OPTIMISATION SANS CONTRAINTE

(Unconstrained Optimization)

Cao Nguyen DOAN
Yonghan LIAO
Oscar CONAN
Alexia MALLET
Basile JÉZÉQUEL

Exercice 1. Quadratic optimization problems
Exercice 2. Gradient type algorithms
Exercice 3. Extensions to unconstrained nonlinear problems
Exercice 4. Second order Newton type methods

EXERCICE 1. PROBLÈME DE L'OPTIMISATION QUADRATIQUE

On considère le problème:

$$\min \quad J(x) := \frac{1}{2} x' Ax - b' x + c \quad \text{avec} \quad x \in R^n \qquad \text{(P)}$$

où A est une matrice n x n, $b \in R^n$ et $c \in R^n$

1. Supposons que A est symétrique, montrons que $\nabla J(x) = Ax - b$

Tout d'abord, nous pouvons réécrire J(x) sous forme:

$$J(x) = \frac{1}{2} \langle x, Ax \rangle - \langle b, x \rangle + c$$

Soit $h \in R^n$ une direction, soit t ≠ 0, nous voulons calculer la dérivée de J. Pour le faire, il nous faut séparer J en deux parties: $J_1 = \langle x, Ax \rangle$ et $J_2 = \langle b, x \rangle$

- ✔ On commence par J_1 :

$$\begin{aligned} J_1(x+th) - J_1(x) &= \langle x+th, A(x+th) \rangle - \langle x, Ax \rangle \\ &= \langle x+th, Ax \rangle + t \langle x+th, Ah \rangle - \langle x, Ax \rangle \\ &= \langle x, Ax \rangle + t \langle h, Ax \rangle + t \langle x, Ah \rangle + t^2 \langle h, Ah \rangle - \langle x, Ax \rangle \\ &= t \langle h, Ax \rangle + t \langle x, Ah \rangle + t^2 \langle h, Ah \rangle \end{aligned}$$

Comme A est ***symétrique***, alors $\langle h, Ax \rangle = \langle Ax, h \rangle = \langle x, Ah \rangle$. De l'équation précédente, nous avons donc:

$$DJ_1(x)(h) = \frac{J_1(x+th) - J_1(x)}{t} \rightarrow \langle 2Ax, h \rangle \quad \text{quand t tend vers 0}$$

Choisissons h comme une base de R^n , nous avons alors $\langle \nabla J_1(x), h \rangle = \langle 2Ax, h \rangle$. Nous pouvons en déduire que $\boxed{\nabla J_1(x) = 2Ax}$.

J_1 est bien continue et différentiable avec le gradient $\nabla J_1(x) = 2Ax$.

- ✔ De même façon, nous faisons le calcul avec J_2 :

$$J_2(x+th) - J_2(x) = \langle b, x+th \rangle - \langle b, x \rangle = \langle b, x \rangle + t \langle b, h \rangle - \langle b, x \rangle = t \langle b, h \rangle$$

$$DJ_2(x)(h) = \frac{J_2(x+th) - J_2(x)}{t} \rightarrow \langle b, h \rangle \quad \text{quand t tend vers 0}$$

Nous obtenons que J_2 est continue et différentiable avec le gradient $\boxed{\nabla J_2(x) = b}$.

Puisque $J=\frac{1}{2}J_1-J_2+c$, nous avons donc $\boxed{\nabla J(x)=Ax-b}$

2. Supposons maintenant que ce soit le même que celui de la question 1. De plus, étant donné qu'il a au moins une valeur propre négative.

Nous cherchons maintenant la matrice Hessienne de J. D'après le 1, nous pouvons en déduire que $\nabla^2 J(x)=A$.

La matrice A a au moins une valeur propre négative. Cela traduit le fait que pour n'importe quelle solution x_0 de $\nabla J(x)=Ax-b=0$, la matrice $\nabla^2 J(x_0)$ n'est pas semi-définie positive. x_0 ne peut pas être une solution de minimisation dans ce cas.

Il n'y a pas donc de solution d'optimisation.

3. Supposons cette fois-ci que A est symétrique définie positive.

D'après le 2, $\nabla^2 J(x)=A$ est symétrique définie positive. De plus, A est inversible, alors l'équation $Ax-b=0$ a une unique solution.

Conclusion: le problème a une unique solution.

TP2 exercice 2

March 31, 2021

0.1 EXERCICE 2

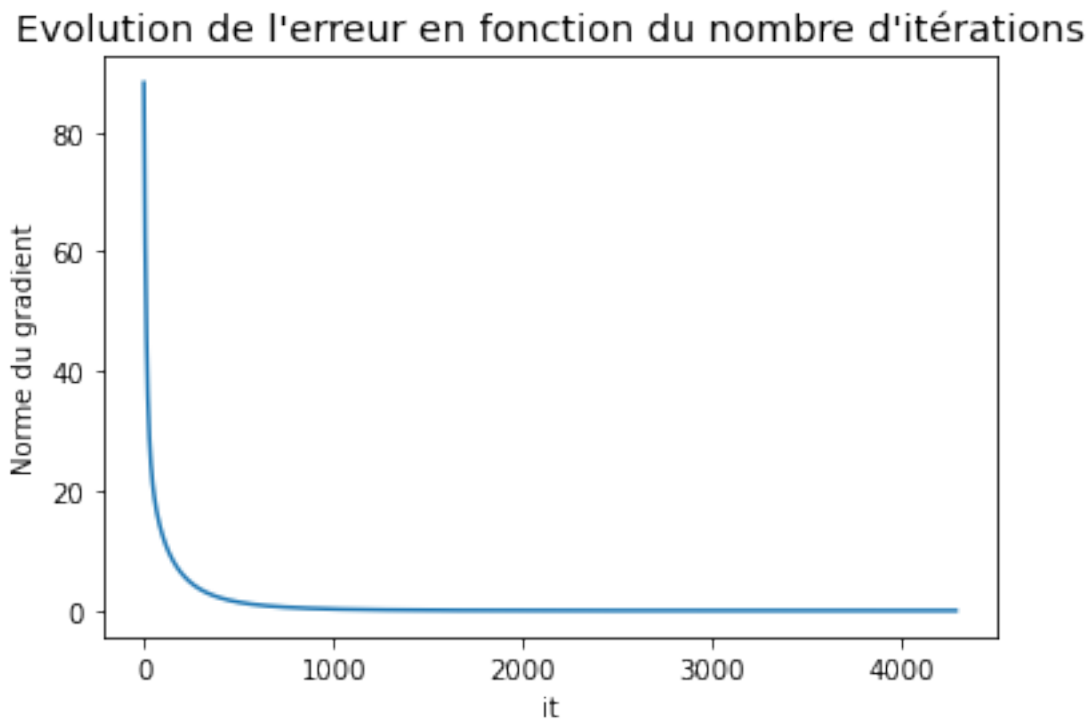
Le but de cet exercice est d'étudier 3 différents types de méthodes de gradient, et ce qui peut être mis en place afin d'avoir une convergence plus rapide.

[95]:
```
#Importation des librairies utiles

import numpy as np
import time as time
import matplotlib.pyplot as plt
import pandas as pd
import scipy as sp

#Méthode du gradient à pas constant :

def GMC(A,b,x0,rho=0.001,tol=1e-4,itmax=10000):
    x=x0
    it=0
    g=np.dot(A,x)-b
    err=np.linalg.norm(g)
    vecterr=np.array(err)
    while (err>tol) and (it<itmax):
        x=x-rho*g
        g=np.dot(A,x)-b
        err=np.linalg.norm(g)
        vecterr=np.append(vecterr,err)
        it=it+1
    v=0.5*np.dot(np.dot(x.T,A),x)-np.dot(b.T,x)
    return x,v,err,vecterr,it


#Définition du problème

n=20
A=4*(np.random.random((n,n))-0.5)
b=10*(np.random.random(n)-0.5)
A=np.dot(A.T,A)+2*np.eye(n)
x0=np.ones(n)
```

```
print("Il y a convergence en : %d itérations" %GMC(A,b,x0)[4])
plt.plot(GMC(A,b,x0)[3])
plt.xlabel("it")
plt.ylabel("Norme du gradient")
plt.title("Evolution de l'erreur en fonction du nombre␣
↪d'itérations",fontsize="x-large")
```

```
Il y a convergence en : 4283 itérations
```

[95]: Text(0.5, 1.0, "Evolution de l'erreur en fonction du nombre d'itérations")

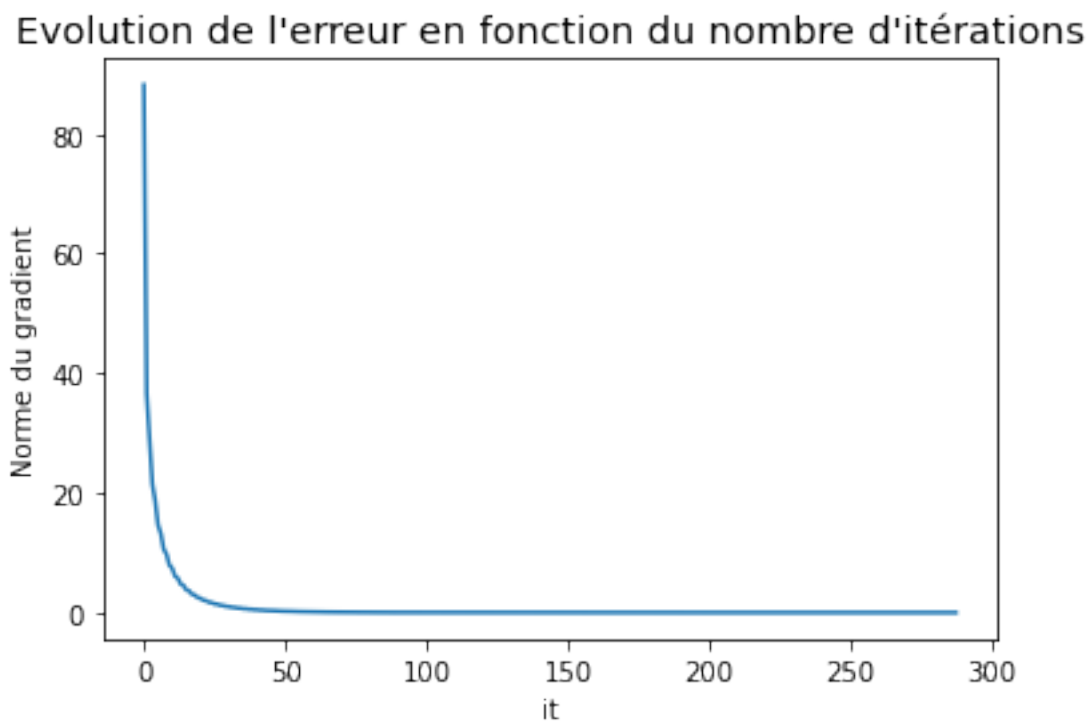


[96]:
```
#Méthode du gradient à pas otpimal :

def GMO(A,b,x0,tol=1e-6,itmax=10000):
    x=x0
    it=0
    g=np.dot(A,x)-b
    err=np.linalg.norm(g)
    vecterr=np.array(err)
    #t1=time.clock()
    while (err>tol) and (it<itmax):
```

```
        rho=np.dot(g.T,g)/np.dot(np.dot(g.T,A),g)
        x=x-rho*g
        g=np.dot(A,x)-b
        err=np.linalg.norm(g)
        vecterr=np.append(vecterr,err)
        it=it+1
    v=0.5*np.dot(np.dot(x.T,A),x)-np.dot(b.T,x)
    #t2=time.clock()
    #temps=t2-t1
    return x,v,err,vecterr,it


print("Il y a convergence en : %d itérations" %GMO(A,b,x0)[4])
plt.plot(GMO(A,b,x0)[3])
plt.xlabel("it")
plt.ylabel("Norme du gradient")
plt.title("Evolution de l'erreur en fonction du nombre␣
 ↪d'itérations",fontsize="x-large")
```

```
Il y a convergence en : 287 itérations
```

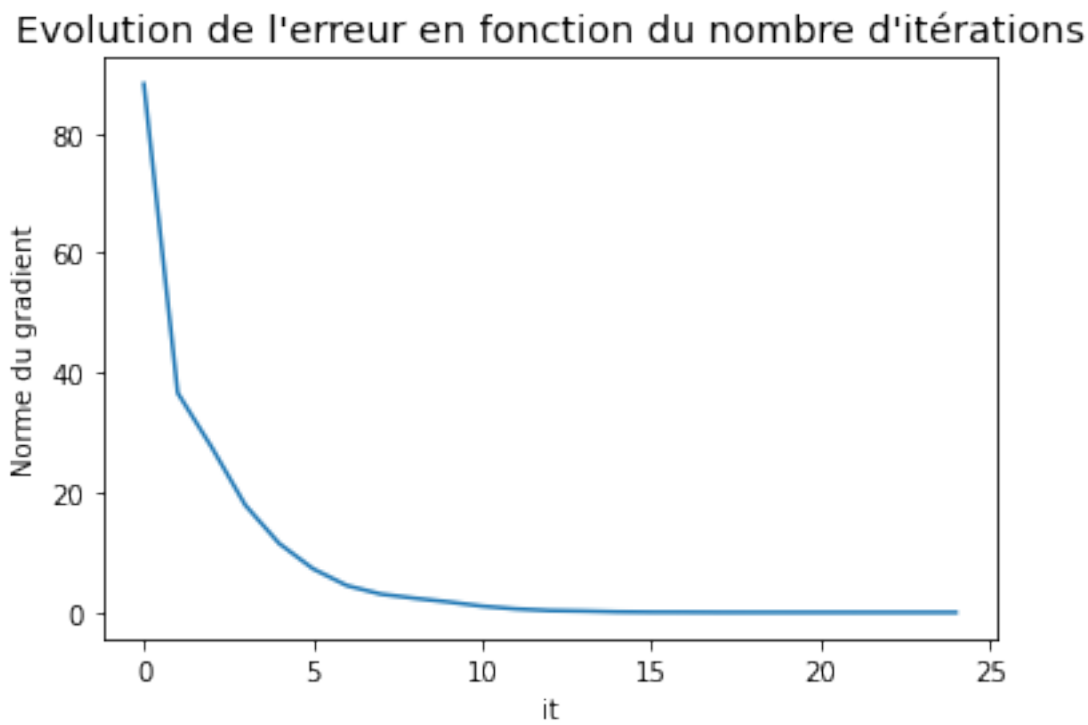
[96]: Text(0.5, 1.0, "Evolution de l'erreur en fonction du nombre d'itérations")



[97]:
```
#Méthgode du gradient conjugué

def CGM(A,b,x0,tol=1e-10,itmax=10000):
    x=x0
    it=0
    r=np.dot(A,x)-b
    d=-r
    err=np.linalg.norm(r)
    vecterr=np.array(err)
    while (err>tol) and (it<itmax):
        it=it+1
        rho=np.dot(r.T,r)/np.dot(np.dot(d.T,A),d)
        x=x+rho*d
        r0=r
        r=np.dot(A,x)-b
        beta=np.dot(r.T,r)/(np.dot(r0.T,r0))
        d=-r+beta*d
        err=np.linalg.norm(r)
        vecterr=np.append(vecterr,err)
    v=0.5*np.dot(np.dot(x.T,A),x)-np.dot(b.T,x)
    return x,v,err,vecterr,it

print("Il y a convergence en : %d itérations" %CGM(A,b,x0)[4])
plt.plot(CGM(A,b,x0)[3])
plt.xlabel("it")
plt.ylabel("Norme du gradient")
plt.title("Evolution de l'erreur en fonction du nombre␣
 ↪d'itérations",fontsize="x-large")
```

```
Il y a convergence en : 24 itérations
```

[97]:
```
Text(0.5, 1.0, "Evolution de l'erreur en fonction du nombre d'itérations")
```



[98]:
```
#Construction de la nouvelle matrice A symétrique, et du vecteur b :

def Construct(n):
    b=np.ones(n)
    v=-2*np.ones(n-1)
    A=4*np.diag(b)+np.diag(v,1)+np.diag(v,-1)
    A[0,n-1]=-1
    A[n-1,0]=-1
    return A,b

#Est-elle définie positive ?

if(min(np.linalg.eig(A)[0])>0):
    print("La matrice est bien définie positive")

#Utilisation des trois méthodes :

n=10
A=Construct(n)[0]
x0=np.zeros(n)
b=Construct(n)[1]
tol=1e-10
rho=0.1
```

```
itmax=100


[x_GMC,v_GMC,err_GMC,vec_GMC,it_GMC]=GMC(A,b,x0,rho,tol,itmax)
[x_GMO,v_GMO,err_GMO,vec_GMO,it_GMO]=GMO(A,b,x0,tol,itmax)
[x_CGM,v_CGM,err_CGM,vec_CGM,it_CGM]=CGM(A,b,x0,tol,itmax)

x=np.array([[it_GMC,err_GMC],[it_GMO,err_GMO],[it_CGM,err_CGM]])
pd.DataFrame(x,index=["GMC","GMO","CGM"],columns=["Nombre␣
↪d'itérations","Erreur"])
```

```
La matrice est bien définie positive
```

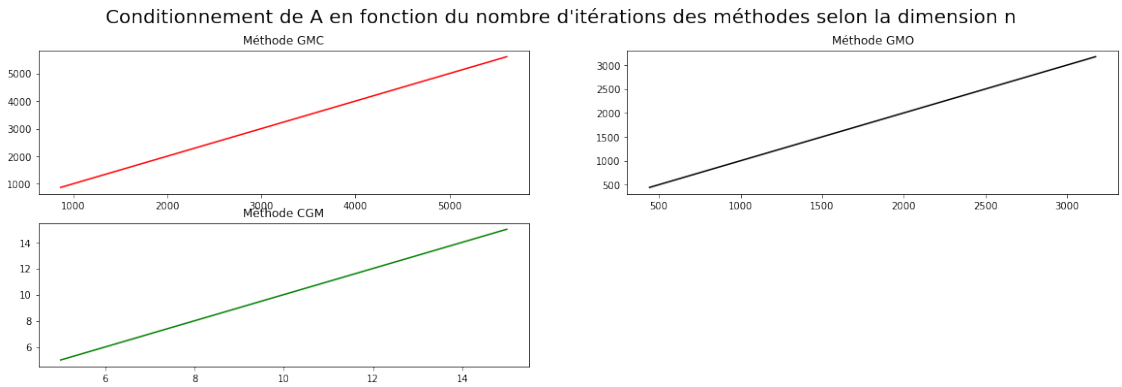
[98]:

	Nombre d'itérations	Erreur
GMC	100.0	0.933420
GMO	100.0	0.073028
CGM	5.0	0.000000

[123]:

```
#Conditionnement de A en fonction du nombre d'itérations

cond_GMC=item_GMC=np.ones(20)
cond_GMO=item_GMO=np.ones(20)
cond_CGM=item_CGM=np.ones(20)
i=0

#Pour différentes dimensions, regardons si le conditonnement de A joue un rôle␣
↪sur le nombre d'itérations des méthodes :

for n in np.arange(10,30):
    A=Construct(n)[0]
    b=Construct(n)[1]
    x0=np.zeros(n)
    #GMC
    cond_GMC[i]=np.linalg.cond(A,2)
    item_GMC[i]=GMC(A,b,x0,rho=0.1)[4]
    #GMO
    cond_GMO[i]=np.linalg.cond(A,2)
    item_GMO[i]=GMO(A,b,x0)[4]
    #CGM
    cond_CGM[i]=np.sqrt(np.linalg.cond(A,2))
    item_CGM[i]=CGM(A,b,x0)[4]
    i=i+1

fig=plt.figure(figsize=[20,6])
fig.suptitle("Conditionnement de A en fonction du nombre d'itérations des␣
↪méthodes selon la dimension n",fontsize=20)
ax1=fig.add_subplot(221)
```

```
ax1.title.set_text("Méthode GMC")
ax1=plt.plot(item_GMC,cond_GMC,"r")
ax2=fig.add_subplot(222)
ax2.title.set_text("Méthode GMO")
ax2=plt.plot(item_GMO,cond_GMO,"k")
ax3=fig.add_subplot(223)
ax3.title.set_text("Méthode CGM")
ax3=plt.plot(item_CGM,cond_CGM,"g")
```



On remarque bien que le conditionnement de A à un effet sur le nombre d'itérations de chaque méthode : il augmente. Pour remédier à cela nous allons voir comment nous pouvons réduire ce conditionnement afin de converger vers la solution plus rapidement.

[125]:
```
#Nouvelle matrice A symétrique:

def Construct2(n):
    v=-np.ones(n-1)
    A=np.diag(3*(np.arange(1,n+1)**2)*np.ones(n).T)+np.diag(v,1)+np.diag(v,-1)
    return A

#Matrice CAC :

def Construct_CAC(n):
    #C(i,i)=1/i matrice diagonale
    C=np.round(np.diag(1/np.linspace(1,n,n)),2)
    A=Construct2(n)
    b=np.ones(n)
    CAC=np.dot(C,np.dot(A,C))
    Cb=np.dot(C,b)
    return CAC,Cb

#Conditonnement selon la valeur de n :
```

```
cond_A=[]
cond_CAC=[]
for n in [10,100,1000]:
    A=Construct2(n)
    CAC=Construct_CAC(n)[0]
    cond_A=np.append(cond_A,np.linalg.cond(A,2))
    cond_CAC=np.append(cond_CAC,np.linalg.cond(CAC,2))



cond=pd.DataFrame([cond_A,cond_CAC],index=["Matrice A","Matrice␣
↪CAC"],columns=["10","100","1000"])
cond.index.name="Conditionnement"
cond.columns.name="n"
cond
```

[125]:

n	10	100	1000
Conditionnement			
Matrice A	103.821952	10381.588747	1.038159e+06
Matrice CAC	1.427236	3.881490	inf

En construisant la matrice CAC, on remarque bien son conditionnement reste très bas bien que la dimension augmente, cela devrait nous permettre de converger vers la solution plus rapidement.

On passe donc du problème : $Ay = b$ avec $y = Cx$, à $CACx = Cb$ avec $y = Cx$, ce qui revient au même :

$ACx = b \Leftrightarrow CACx = Cb$

Résolvons alors avec la méthode du gradient conjugué et comparons les nombres d'itération et les erreurs.

[126]:

```
#Test de la méthode du gradient conjugué pour A et CAC :

n=1000
itmax=2000
tol=1e-10

A=Construct2(n)
b=np.ones(n)

x0=np.ones(n)

CAC=Construct_CAC(n)[0]
Cb= Construct_CAC(n)[1]


[xA_CGM,vA_CGM,errA_CGM,vecA_CGM,itA_CGM]=CGM(A,b,x0,tol,itmax)
[xCAC_CGM,vCAC_CGM,errCAC_CGM,vecCAC_CGM,itCAC_CGM]=CGM(CAC,Cb,x0,tol,itmax)
```

```
data_CGM=pd.DataFrame(np.
↪array([[vecA_CGM[-1],itA_CGM],[vecCAC_CGM[-1],itCAC_CGM]]),index=["A","CAC"],columns=["Erre
↪d'itérations"])
data_CGM.index.name="Méthode CGM"
data_CGM
```

[126]:

Méthode CGM	Erreur	Nombre d'itérations
A	7.635653e-11	1545.0
CAC	5.740801e-11	39.0

Avec la matrice CAC, le nombre d'itérations est considérablement réduit : on passe de 1545 à 39. Ce qui réduit évidemment le temps d'éxécution aussi.

0.2 Utilisation de préconditionneurs

La méthode utilisée juste au dessus pour réduire le conditionnement de la matrice, est le préconditonnement.

Il existe plusieurs méthode de préconditionnement. (matrice P telle qu'on résoud le systême suivant : $P^{-1}Ax = P^{-1}b$.

Cela revient à, comme dans la question précédente, résoudre : $P^{-1}Ax = P^{-1}b$ au lieu de $Ax = b$.

Le préconditionneur est la matrice P, et doit être, tout comme A, symétrique et définie positive.

Nous allons étudier deux exemples :

-Le préconditionneur de Jacobi $P = diag(A)$

-Le préconditionneur SGS : $A = D - E - F$ on construit $P = (D - E)D^{-1}(D - F)$

Ces deux préconditonneurs sont bien symétriques et définies positives si A l'est.

[8]:

```
#Utilisation de préconditionneur pour réduire le conditionnement de la matrice␣
↪dans l'algorithme :

#Ici nous allons utiliser le préconditionneur de Jacobi : P=diag(A) :

n=1000
A=Construct(n)[0]

#Matrice A à coefficent constants :

P=np.diag(A)*np.eye(n)
P_inv=np.linalg.inv(P)
A_J=np.dot(P_inv,A)
#Comparaison du conditionnement
```

```
print("Conditionnement de A :",np.linalg.cond(A),"\n","Conditionnement de A_J :
 ↪",np.linalg.cond(A_J))
```

```
Conditionnement de A : 407719.4079107178
 Conditionnement de A_J : 407719.4079107178
```

On remarque que pour les matrices à coefficients constants, le préconditionneur de Jacobi n'apporte rien, le conditionnement est identique. Étudions une autre matrice :

[127]:
```
#Construction d'une nouvelle matrice exemple

def Construct3(n):
    v=np.ones(n-2)
    A=np.diag(np.arange(0,n)+1)*np.ones(n).T+np.diag(v,2)+np.diag(v,-2)
    return A

n=500
A=Construct3(n)
b=np.ones(n)
x0=np.ones(n)
tol=1e-10
itmax=5000


#Est-elle définie positive ?

if(min(np.linalg.eig(A)[0])>0):
    print("La matrice est bien définie positive")


#Préconditionneur de Jacobi :

P=np.diag(A)*np.eye(n)
P_inv=np.linalg.inv(P)

A_J=np.dot(P_inv,A)
b_J=np.dot(P_inv,b)

#Comparaison du conditionnement

print("Conditionnement de A :",np.linalg.cond(A),"\n","Conditionnement de␣
 ↪P_in*vA :",np.linalg.cond(A_J))


[xA3_CGM,vA3_CGM,errA3_CGM,vecA3_CGM,itA3_CGM]=CGM(A,b,x0,tol,itmax)
[xJ_CGM,vJ_CGM,errJ_CGM,vecJ_CGM,itJ_CGM]=CGM(A_J,b_J,x0,tol,itmax)
```

```
data_CGM=pd.DataFrame(np.
 ↪array([[errA3_CGM,itA3_CGM],[vecJ_CGM[-1],itJ_CGM]]),index=["A","P_inv*A"],columns=["Erreur
 ↪d'itérations"])
data_CGM.index.name="Méthode CGM"
data_CGM
```

```
La matrice est bien définie positive
Conditionnement de A : 911.353885086241
 Conditionnement de P_in*vA : 5.355979649385164
```

[127]:

Méthode CGM	Erreur	Nombre d'itérations
A	9.165151e-11	165.0
P_inv*A	9.985281e-11	92.0

Nous pouvons observer que la méthode de gradient à poas conjugué converge bien plus rapidement avec la matrice $P^{-1}A$, le préconditonnement fonctionne bien.

[128]:

```
#Utilisation du pérconditonneur SGS :

A=Construct3(n)

#Construction de la méthode : A=D-E-F, avec P=(D-E)D-1(D-F)
D=np.diag(A)*np.eye(n)
E=-np.triu(A,k=1)
F=-np.tril(A,k=-1)
b=np.ones(n)
x0=np.ones(n)
tol=1e-10
itmax=5000

#Construction de P :

P=(D-E).dot((np.linalg.inv(D)).dot(D-F))
P_inv=np.linalg.inv(P)
A_SGS=np.dot(P_inv,A)
b_SGS=np.dot(P_inv,b)

#Comparaison du conditionnement

print("Conditionnement de A :",np.linalg.cond(A),"\n","Conditionnement de␣
 ↪P_in*vA :",np.linalg.cond(A_SGS))


[xA4_CGM,vA4_CGM,errA4_CGM,vecA4_CGM,itA4_CGM]=CGM(A,b,x0,tol,itmax)
[xSGS_CGM,vSGS_CGM,errSGS_CGM,vecSGS_CGM,itSGS_CGM]=CGM(A_SGS,b_SGS,x0,tol,itmax)
```

```
data_CGM=pd.DataFrame(np.
 ↪array([[errA4_CGM,itA4_CGM],[vecSGS_CGM[-1],itSGS_CGM]]),index=["A","P_inv*A"],columns=["Er
 ↪d'itérations"])
data_CGM.index.name="Méthode CGM"
data_CGM
```

```
Conditionnement de A : 911.353885086241
 Conditionnement de P_in*vA : 1.5701094728745686
```

[128]:

Méthode CGM	Erreur	Nombre d'itérations
A	9.165151e-11	165.0
P_inv*A	2.331586e-11	11.0

Pour ce préconditionneur, c'est encore plus flagrant : le nombre d'itération est très réduit, et le conditionnement de $P^{-1}A$ est proche de l'idéal : 1.

En conclusion,

Pour résoudre les problèmes d'optimisation, nous pouvons utiliser ces différents algorithmes (problèemes quadratiques pour les deux derniers) en veillant à ce que le conditionnement des matrices utilisées n'est pas trés haut. Pour remédier à cela si c'est le cas, nous pouvons utiliser les techniques de préconditonnement qui permettront de résoudre un système équivalent bien plus rapidement, pour une erreur du même ordre.

[ ]:

```
```

TP2 exercice 3

March 31, 2021

0.1 Exercice 3

0.2 S.D.M : Steepest Descent Method

[57]:
```
# Méthode d'Armijo pour trouver le pas
# f est continue et différentiable

def Armijo(f,g0,x,d,t=1,beta=0.5,alpha=0.25,itmax=10000):
    it=0
    while (f(x+t*d)>f(x)+alpha*t*np.dot(g0,d)) and (it<itmax):
        it+=1
        t=beta*t
    return t, f(x+t*d), it
```

[58]:
```
import numpy as np

def SDM(f,gradf,x, Tol=1e-10, itmax=10000):
    it=0
    vect=[0]
    while (np.linalg.norm(gradf(x))>Tol) and (it<itmax) :
        d=-gradf(x)
        rho=Armijo(f,gradf(x),x,d)[0]
        x=x+rho*d
        err=np.linalg.norm(gradf(x))
        vect=np.append(vect,err)
        it+=1
    return x, gradf(x), it, vect
```

0.3 F.R : Fletcher Reeves Method

[59]:
```
def FR(f,gradf,x,Tol=1e-10,itmax=10000):
    it=0
    vect=[0]
    r=gradf(x)
    d=-r
    while (np.linalg.norm(gradf(x))>Tol) and (it<itmax) :
        rho=Armijo(f,gradf(x),x,d)[0]
        x=x+rho*d
```

```
        r=gradf(x)
        beta=(np.linalg.norm(r)/np.linalg.norm(d))**2
        d=-r+beta*d
        it+=1
        err=np.linalg.norm(r)
        vect=np.append(vect,err)
    return x, r,it,vect
```

0.4 Fonction de comparaison des deux algorithmes

[60]:
```
# On teste n fois les deux algorithmes

# La fonction retourne pour chaque algorithme le nombre de fois où il n'a pas␣
↪convergé sur n
# ainsi que le nombre d'itération moyen qu'il a fallu
def comparaison(n,f,gradf):
    itmax = 10000
    pas_de_convergence_SDM = 0
    pas_de_convergence_FR = 0
    average_it_SDM,average_it_FR = 0,0
    for _ in range(n):
        X = np.random.sample(2)
        it = SDM(f,gradf,X,1e-6,itmax)[2]
        if it == itmax: pas_de_convergence_SDM += 1
        average_it_SDM += it

        it = FR(f,gradf,X,1e-6,itmax)[2]
        if it == itmax: pas_de_convergence_FR += 1
        average_it_FR += it

    print("l algorithme SDM n'a pas convergé : "+str(pas_de_convergence_SDM)+"/
↪"+str(n))
    print("nombre moyen d itérations pour SDM : "+str(round(average_it_SDM/n)))
    print("")

    print("l algorithme FR n'a pas convergé : "+str(pas_de_convergence_FR)+"/
↪"+str(n))
    print("nombre moyen d itérations pour FR : "+str(round(average_it_FR/n)))
```

1 Test avec différentes fonctions

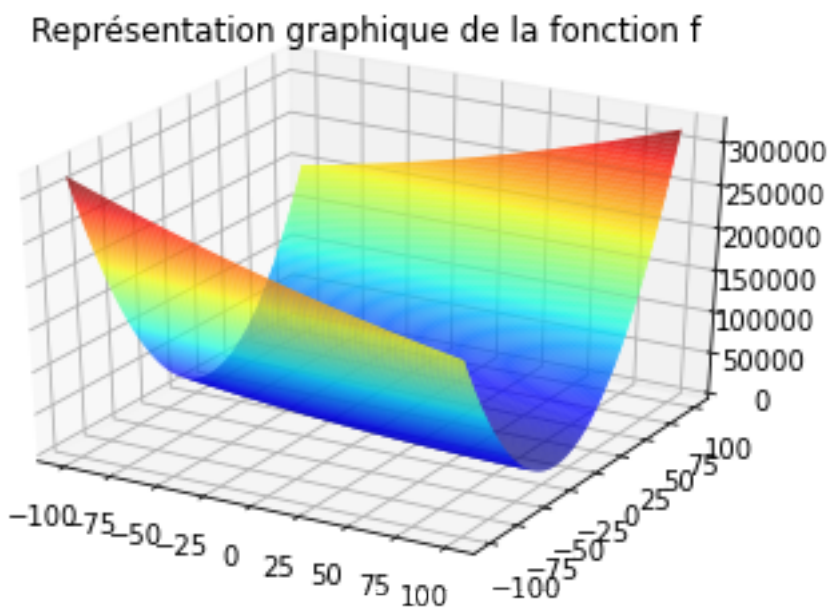
1.1 f(x)=x1^2+6*x1*x2+25*x2^2-12*x1-2*x2-6

[61]:
```
def func_un(x):
    x1=x[0]
    x2=x[1]
    f=x1**2+6*x1*x2+25*(x2**2)-12*x1-2*x2-6
    return f

def gradf_un(x):
    x1=x[0]
    x2=x[1]
    gradf=np.array([2*x1+6*x2-12,6*x1+50*x2-2])
    return gradf
```

Représentation graphique

[62]:
```
# Représentation graphique de la fonction f
import numpy as np
import matplotlib.pyplot as plt

xmin = -100
xmax = 100
nbx = 200
ymin = -100
ymax = 100
nby = 200

x = np.linspace(xmin, xmax, nbx)
y = np.linspace(ymin, ymax, nby)
X, Y = np.meshgrid(x, y)

x=np.array([X,Y])

Z = func_un(x)

fig = plt.figure()
ax = plt.axes(projection='3d')
ax.plot_surface(X, Y, Z, rstride=1, cstride=1,cmap='jet', edgecolor='none')
ax.set_title("Représentation graphique de la fonction f")
```

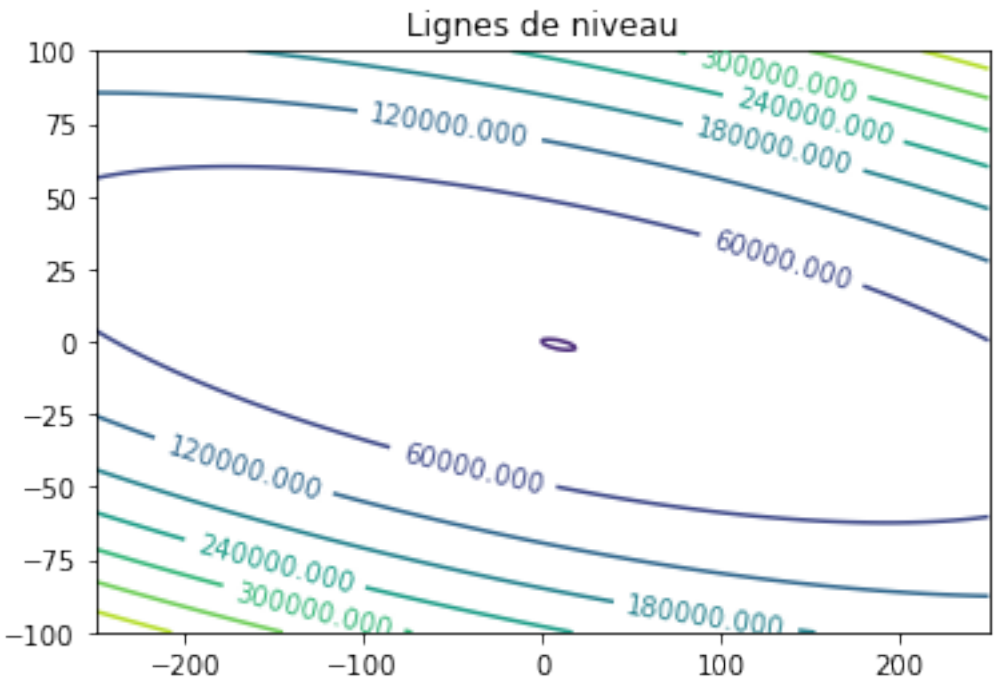
[62]: Text(0.5, 0.92, 'Représentation graphique de la fonction f')



Lignes de niveau

[63]:
```
# Allure des lignes de niveau de la fonction f
import numpy as np
import matplotlib.pyplot as plt

xmin = -250
xmax = 250
nbx = 200
ymin = -100
ymax = 100
nby = 200

x = np.linspace(xmin, xmax, nbx)
y = np.linspace(ymin, ymax, nby)
X, Y = np.meshgrid(x, y)

x=np.array([X,Y])

Z = func_un(x)

fig, ax = plt.subplots()
CS = ax.contour(X, Y, Z)
ax.clabel(CS, inline=True, fontsize=10)
ax.set_title('Lignes de niveau')
```

[63]: Text(0.5, 1.0, 'Lignes de niveau')



Résultats obtenus avec l'algorithme SDM

[64]:
```
# Test de l'algorithme SDM
np.random.seed(10) # Pour pouvoir reproduire sans changement
x=np.random.sample(2) # random number
x,gradf,it,vect=SDM(func_un,gradf_un,x)
print ("Résulat de l'algorithme SDM : x=",x)
print ("avec un gradient de norme %f, après %d itération et une erreur %f" %(np.
 ↪linalg.norm(gradf),it,vect[-1]))
```

```
Résulat de l'algorithme SDM : x= [ 9.18749928 -1.0624999 ]
avec un gradient de norme 0.000001, après 10000 itération et une erreur 0.000001
```

Le nombre d'itérations maximales est atteinte donc l'algorithme ne s'est pas arrété après avoir convergé. Cependant, l'erreur ne semble pas très importante. On peut alors essayer avec une tolérance un peu plus importante pour voir si l'algorithme converge.

[65]:
```
np.random.seed(10)
x=np.random.sample(2) # random number
x,gradf,it,vect=SDM(func_un,gradf_un,x,Tol=1e-5)
print ("Résulat de l'algorithme SDM : x=",x)
print ("avec un gradient de norme %f, après %d itération et une erreur %f" %(np.
 ↪linalg.norm(gradf),it,vect[-1]))
```

```
Résulat de l'algorithme SDM : x= [ 9.18749329 -1.06249908]
avec un gradient de norme 0.000010, après 248 itération et une erreur 0.000010
```

On voit ici que l'algorithme s'arrete avant d'avoir fait le nombre d'itération maximum. Il converge donc.

Résultats obtenus avec l'alorithme FR

[66]:
```
# Test de l'algorithme FR
np.random.seed(10)
x=np.random.sample(2) # random number
x,r,it,vect=FR(func_un,gradf_un,x)
print ("Résulat de l'algorithme FR : x=",x)
print ("avec un gradient de norme %f, après %d itération et une erreur %f" %(np.
 ↪linalg.norm(r),it,vect[-1]))
```

```
Résulat de l'algorithme FR : x= [ 9.18749999 -1.0625    ]
avec un gradient de norme 0.000000, après 10000 itération et une erreur 0.000000
```

Le nombre d'itérations maximales est atteinte donc l'algorithme ne s'est pas arrété après avoir convergé. Cependant, l'erreur ne semble pas très importante. On peut alors essayer avec une tolérance un peu plus importante pour voir si l'algorithme converge.

[67]:
```
# Avec une tolérance plus importante
np.random.seed(10)
x=np.random.sample(2) # random number
x,r,it,vect=FR(func_un,gradf_un,x,Tol=1e-5)
print ("Résulat de l'algorithme FR : x=",x)
print ("avec un gradient de norme %f, après %d itération et une erreur %f" %(np.
 ↪linalg.norm(r),it,vect[-1]))
```

```
Résulat de l'algorithme FR : x= [ 9.18750026 -1.0625002 ]
avec un gradient de norme 0.000008, après 26 itération et une erreur 0.000008
```

On voit ici que l'algorithme s'arrete avant d'avoir fait le nombre d'itération maximum. Il converge donc.

Comparaison des deux algorithmes

[68]:
```
comparaison(50,func_un,gradf_un)
```

```
l algorithme SDM n'a pas convergé : 0/50
nombre moyen d itérations pour SDM : 296

l algorithme FR n'a pas convergé : 0/50
nombre moyen d itérations pour FR : 32
```

Ici on remarque que la méthode de Fletcher (FR) converge en moyenne plus rapidemment que SDM (moins d'itérations nécessaires).

Les deux algorithmes convergent.

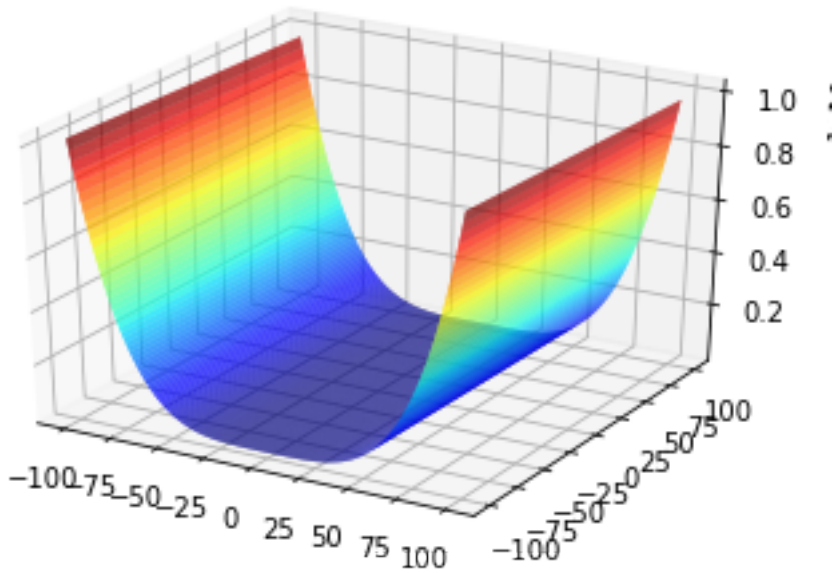
1.2 Fonction de Rosenbrock

[69]:
```
def Rosenbrock(x):
    return 100*(x[1]-x[0]**2)**2+(1-x[0])**2

def grad_Ros(x):
    gradf=np.array([400*x[0]*(-x[1]+x[0]**2)+2*(x[0]-1),200*(x[1]-x[0]**2)])
    return gradf
```

Représentation graphique

[70]:
```
import numpy as np
import matplotlib.pyplot as plt

xmin = -100
xmax = 100
nbx = 200
ymin = -100
ymax = 100
nby = 200

x = np.linspace(xmin, xmax, nbx)
y = np.linspace(ymin, ymax, nby)
X, Y = np.meshgrid(x, y)

x=np.array([X,Y])

Z = Rosenbrock(x)

fig = plt.figure()
ax = plt.axes(projection='3d')
ax.plot_surface(X, Y, Z, rstride=1, cstride=1,cmap='jet', edgecolor='none')
```

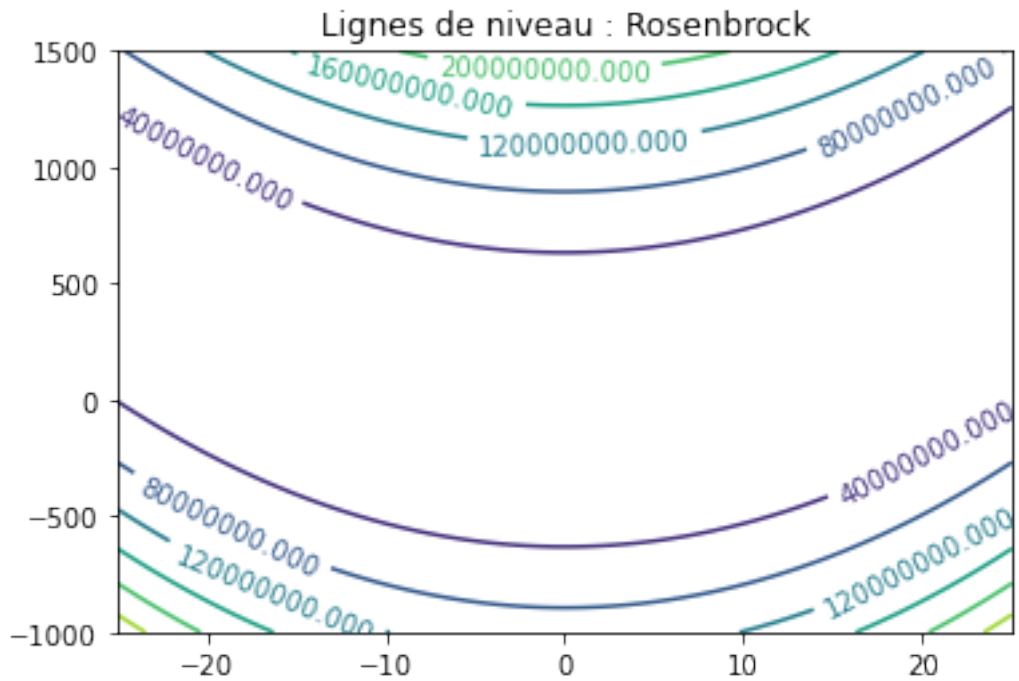
[70]:
```
<mpl_toolkits.mplot3d.art3d.Poly3DCollection at 0x1e1a8222160>
```



Lignes de niveau

[71]:
```
# Allure des lignes de niveau de la fonction Rosenbrook
import numpy as np
import matplotlib.pyplot as plt

xmin = -25
xmax = 25
nbx = 200
ymin = -1000
ymax = 1500
nby = 200

x = np.linspace(xmin, xmax, nbx)
y = np.linspace(ymin, ymax, nby)
X, Y = np.meshgrid(x, y)

x=np.array([X,Y])

Z = Rosenbrock(x)

fig, ax = plt.subplots()
CS = ax.contour(X, Y, Z)
ax.clabel(CS, inline=True, fontsize=10)
ax.set_title('Lignes de niveau : Rosenbrock')
```

[71]: Text(0.5, 1.0, 'Lignes de niveau : Rosenbrock')



Résultats avec l'algorithme SDM

[72]:
```
# Test de l'algorithme SDM
np.random.seed(10)
x=np.random.sample(2) # random number
x,gradf,it,vect=SDM(Rosenbrock,grad_Ros,x)
print ("Résulat de l'algorithme SDM : x=",x)
print ("avec un gradient de norme %f, après %d itération et une erreur %f" %(np.
↪linalg.norm(gradf),it,vect[-1]))
```

```
Résulat de l'algorithme SDM : x= [1. 1.]
avec un gradient de norme 0.000000, après 2443 itération et une erreur 0.000000
```

L'algorithme s'arrête avant le nombre d'itérations maximum fixé, il y a donc convergence.

Résultats avec l'algorithme FR

[73]:
```
# Test de l'algorithme FR
np.random.seed(10)
x=np.random.sample(2) # random number
x,r,it,vect=FR(Rosenbrock,grad_Ros,x)
print ("Résulat de l'algorithme FR : x=",x)
print ("avec un gradient de norme %f, après %d itération et une erreur %f" %(np.
↪linalg.norm(r),it,vect[-1]))
```

```
Résulat de l'algorithme FR : x= [1. 1.]
```

```
avec un gradient de norme 0.000000, après 122 itération et une erreur 0.000000
```

L'algorithme converge également ici.

Comparaison des deux algorithmes

[74]:
```
n=50
comparaison(n,Rosenbrock,grad_Ros)
```

```
l algorithme SDM n'a pas convergé : 0/50
nombre moyen d itérations pour SDM : 2186

l algorithme FR n'a pas convergé : 0/50
nombre moyen d itérations pour FR : 74
```

On observe les mêmes conclusions que pour la première fonction : les deux algorithmes convergent pour une tolérance de 10^-10 mais celui de Fletcher semble plus rapide.

1.3 Fonction Himmelblau

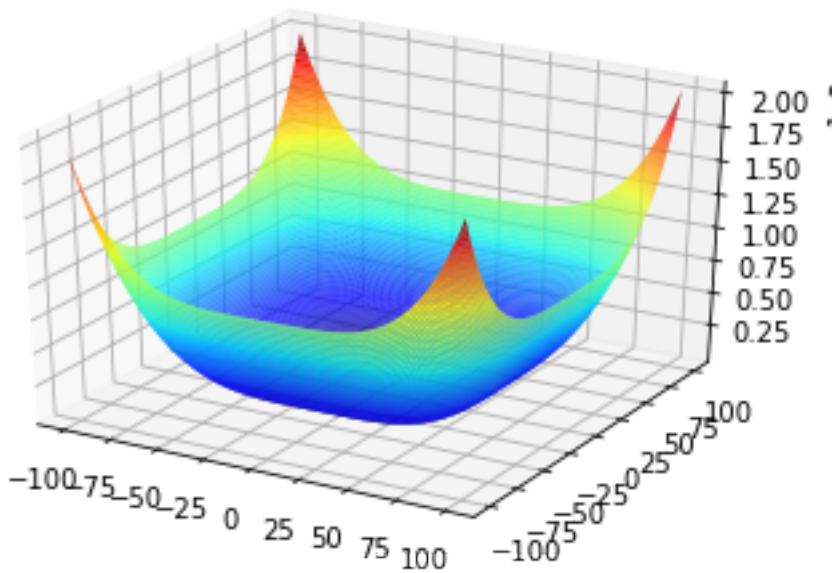
[75]:
```
def Himmelblau(x):
    return (x[1]+x[0]**2-11)**2+(x[0]+x[1]**2-7)**2

def grad_Himmel(x):
    x1=x[0]
    x2=x[1]
    y = np.asarray(x)
    gradf = np.zeros_like(y)
    gradf[0]=4*x1*(x2+x1**2-11)+2*(x1+x2**2-7)
    gradf[1]=2*(x2+x1**2-11)+4*x2*(x1+x2**2-7)
    return gradf
```

Représentation graphique

[76]:
```
# Représentation graphique de la fonction de Himmelblau
import numpy as np
import matplotlib.pyplot as plt

xmin = -100
xmax = 100
nbx = 200
ymin = -100
ymax = 100
nby = 200

x = np.linspace(xmin, xmax, nbx)
y = np.linspace(ymin, ymax, nby)
X, Y = np.meshgrid(x, y)
```

```
x=np.array([X,Y])

Z = Himmelblau(x)

fig = plt.figure()
ax = plt.axes(projection='3d')
ax.plot_surface(X, Y, Z, rstride=1, cstride=1,cmap='jet',edgecolor='none')
```

[76]: <mpl_toolkits.mplot3d.art3d.Poly3DCollection at 0x1e1ab7abe20>

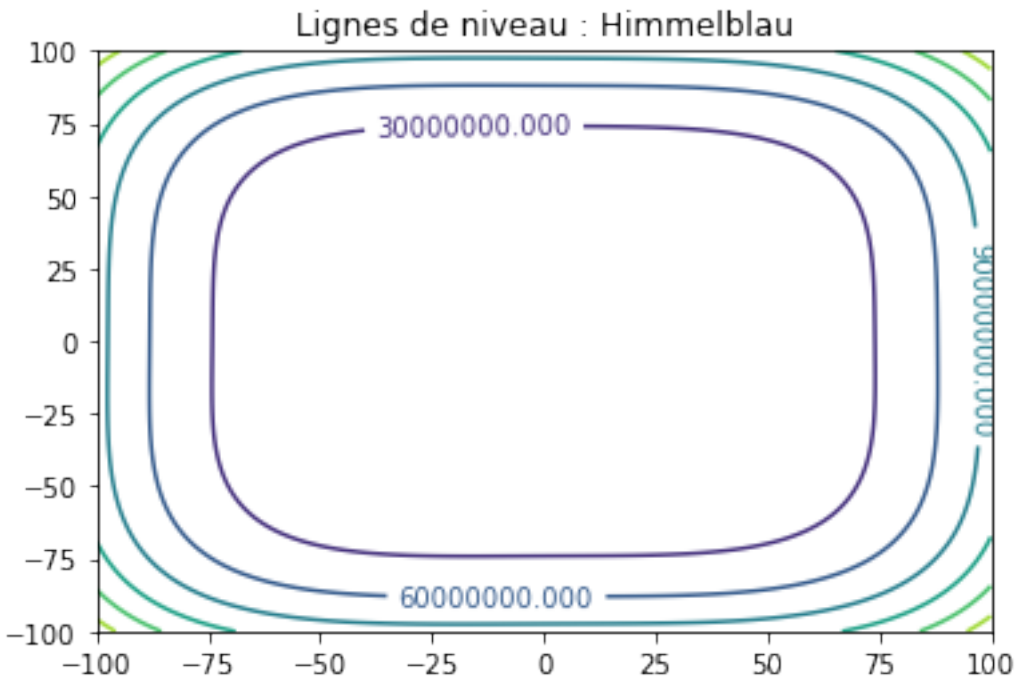


Lignes de niveau

[77]:
```
# Allure des lignes de niveau de la fonction de Himmelblau
import numpy as np
import matplotlib.pyplot as plt

xmin = -100
xmax = 100
nbx = 200
ymin = -100
ymax = 100
nby = 200

x = np.linspace(xmin, xmax, nbx)
y = np.linspace(ymin, ymax, nby)
X, Y = np.meshgrid(x, y)
```

```
x=np.array([X,Y])

Z = Himmelblau(x)

fig, ax = plt.subplots()
CS = ax.contour(X, Y, Z)
ax.clabel(CS, inline=True, fontsize=10)
ax.set_title('Lignes de niveau : Himmelblau')
```

[77]: Text(0.5, 1.0, 'Lignes de niveau : Himmelblau')



Résultats avec l'algorithme SDM

[78]:
```
# Test de l'algorithme SDM
np.random.seed(10)
x=np.random.sample(2) # random number
x,gradf,it,vect=SDM(Himmelblau,grad_Himmel,x)
print ("Résulat de l'algorithme SDM : x=",x)
print ("avec un gradient de norme %f, après %d itération et une erreur %f" %(np.
↪linalg.norm(gradf),it,vect[-1]))
```

```
Résulat de l'algorithme SDM : x= [3. 2.]
avec un gradient de norme 0.000000, après 36 itération et une erreur 0.000000
```

Le nombre d'itérations maximum n'étant pas atteint, l'algorithme ne converge.

[79]:
```
# Test de l'algorithme FR
x=np.random.sample(2) # random number
x,r,it,vect=FR(Himmelblau,grad_Himmel,x)
print ("Résulat de l'algorithme FR : x=",x)
print ("avec un gradient de norme %f, après %d itération et une erreur %f" %(np.
 ↪linalg.norm(r),it,vect[-1]))
```

```
Résulat de l'algorithme FR : x= [3. 2.]
avec un gradient de norme 0.000000, après 35 itération et une erreur 0.000000
```

De même que pour l'algorithme SDM, FR converge.

Comparaison des deux algorithmes

[80]:
```
n=50
comparaison(n,Himmelblau,grad_Himmel)
```

```
l algorithme SDM n'a pas convergé : 0/50
nombre moyen d itérations pour SDM : 22

l algorithme FR n'a pas convergé : 0/50
nombre moyen d itérations pour FR : 23
```

Les deux algorithmes convergent.

La différence entre eux ne semble ici pas flagrante. Le nombre moyen d'itérations des deux algorithmes est assez proche.

1.4 Fonction Denni

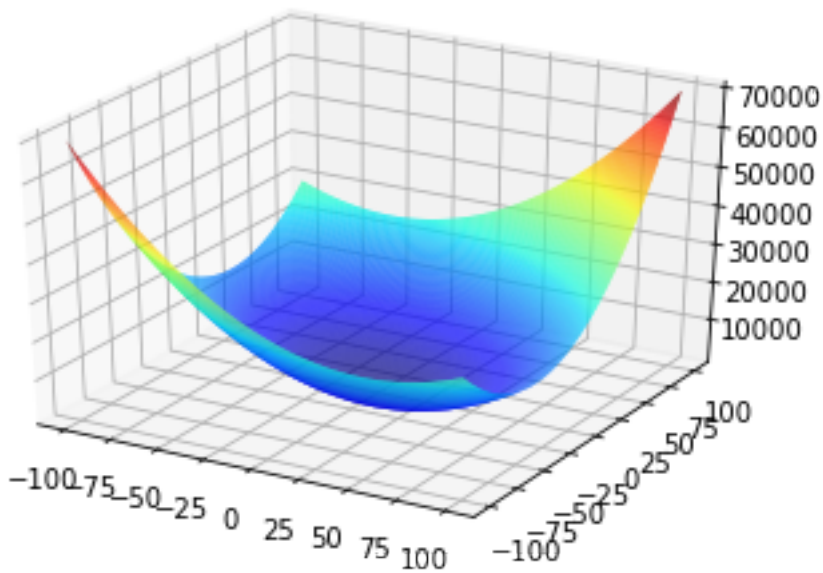
[81]:
```
def Denni(x):
    vect=np.arange(1,np.size(x)+1)
    f=np.sum(vect*x*x)+(np.sum(x))**2
    return f

def grad_Den(x):
    vect=np.arange(1,np.size(x)+1)
    gradf=2*vect*x+2*np.sum(x)
    return gradf
```

Représentation graphique en 3D

[82]:
```
# Fonction Denni de dimension 2
def Denni_dim2(x):
    return x[0]**2+2*x[1]**2+(x[0]+x[1])**2

# Représentation graphique de la fonction Denni
import numpy as np
import matplotlib.pyplot as plt
```

```
xmin = -100
xmax = 100
nbx = 200
ymin = -100
ymax = 100
nby = 200

x = np.linspace(xmin, xmax, nbx)
y = np.linspace(ymin, ymax, nby)
X, Y = np.meshgrid(x, y)

x=np.array([X,Y])

Z = Denni_dim2(x)

fig = plt.figure()
ax = plt.axes(projection='3d')
ax.plot_surface(X, Y, Z, rstride=1, cstride=1,cmap='jet', edgecolor='none')
```

[82]: <mpl_toolkits.mplot3d.art3d.Poly3DCollection at 0x1e1aebf1070>

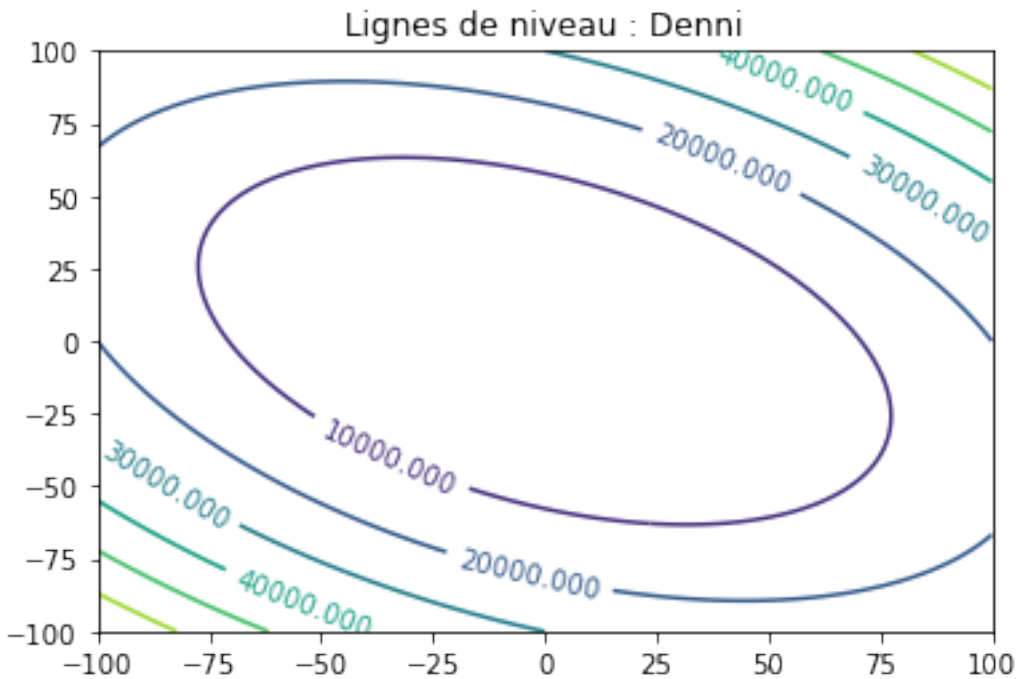


Lignes de niveau

[83]:
```
# Allure des lignes de niveau de la fonction Denni
import numpy as np
import matplotlib.pyplot as plt
```

```
xmin = -100
xmax = 100
nbx = 200
ymin = -100
ymax = 100
nby = 200

x = np.linspace(xmin, xmax, nbx)
y = np.linspace(ymin, ymax, nby)
X, Y = np.meshgrid(x, y)

x=np.array([X,Y])

Z = Denni_dim2(x)

fig, ax = plt.subplots()
CS = ax.contour(X, Y, Z)
ax.clabel(CS, inline=True, fontsize=10)
ax.set_title('Lignes de niveau : Denni')
```

[83]: Text(0.5, 1.0, 'Lignes de niveau : Denni')



Résultats de l'algorithme SDM

[84]:
```
# Test de l'algorithme SDM
n=3 # Dimension du vecteur
x=np.random.sample(n) # random number
x,gradf,it,vect=SDM(Denni,grad_Den,x)
print ("Résulat de l'algorithme SDM : x=",x)
print ("avec un gradient de norme %f, après %d itération et une erreur %f" %(np.
↪linalg.norm(gradf),it,vect[-1]))
```

```
Résulat de l'algorithme SDM : x= [ 9.26961478e-12 -6.96401513e-12
-4.72706314e-12]
avec un gradient de norme 0.000000, après 39 itération et une erreur 0.000000
```

Résultats de l'algorithme FR

[85]:
```
# Test de l'algorithme FR
n=3 # Dimension du vecteur
x=np.random.sample(n) # random number
x,r,it,vect=FR(Denni,grad_Den,x)
print ("Résulat de l'algorithme FR : x=",x)
print ("avec un gradient de norme %f, après %d itération et une erreur %f" %(np.
↪linalg.norm(r),it,vect[-1]))
```

```
Résulat de l'algorithme FR : x= [-4.39454802e-12  6.43305633e-13
1.47874578e-12]
avec un gradient de norme 0.000000, après 25 itération et une erreur 0.000000
```

Comparaison des deux algorithmes

[86]:
```
comparaison(50,Denni,grad_Den)
```

```
l algorithme SDM n'a pas convergé : 0/50
nombre moyen d itérations pour SDM : 15

l algorithme FR n'a pas convergé : 0/50
nombre moyen d itérations pour FR : 17
```

Les deux algorithmes semblent ici converger à une vitesse similaire.

1.5 Fonction VAR

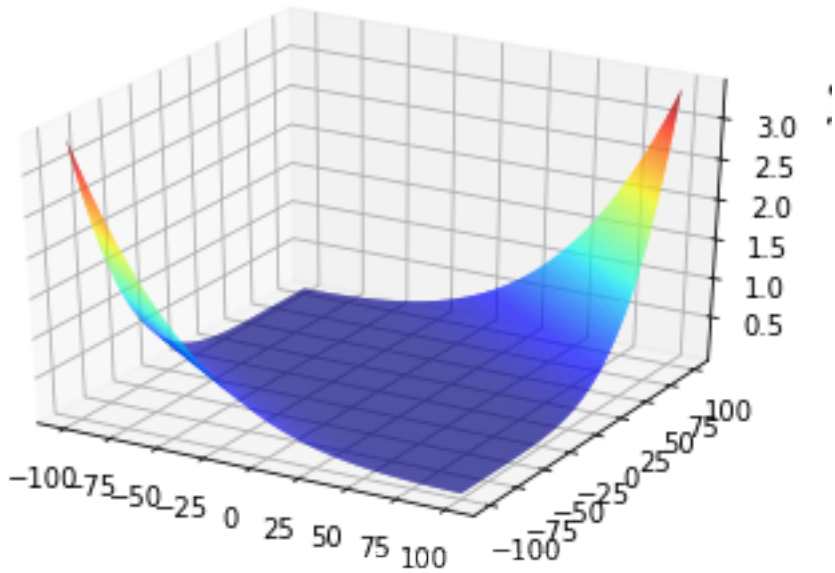
[87]:
```
def VAR(x):
    vect=np.arange(1,np.size(x)+1)
    f=sum(x*x)+(sum(np.sqrt(vect)*x))**2+(sum(np.sqrt(vect)*x))**4
    return f

def grad_Var(x):
    vect=np.arange(1,np.size(x)+1)
    gradf=2*sum(x)+ 2*np.sqrt(vect)*x*(1+2*(sum(np.sqrt(vect)*x))**2)
    return gradf
```

[88]:
```
# Fonction VAR pour une dimension égale à 2
def VAR_dim2(x):
    x1=x[0]
    x2=x[1]
    f=x1**2+x2**2+(x1+np.sqrt(2)*x2)**2+(x1+np.sqrt(2)*x2)**4
    return f

def grad_Var2(x):
    x1=x[0]
    x2=x[1]
    gradf=np.array([2*x1+(2*x1*2*np.sqrt(2)*x2)*(1+2*(x1+np.
 ↪sqrt(2))**2),2*x2+(4*x2+2*np.sqrt(2)*x1)*(1+2*(x1+np.sqrt(2))**2)])
    return gradf
```

Représentation graphique

[89]:
```
# Représentation graphique de la fonction VAR
import numpy as np
import matplotlib.pyplot as plt

xmin = -100
xmax = 100
nbx = 200
ymin = -100
ymax = 100
nby = 200

x = np.linspace(xmin, xmax, nbx)
y = np.linspace(ymin, ymax, nby)
X, Y = np.meshgrid(x, y)

x=np.array([X,Y])

Z = VAR_dim2(x)

fig = plt.figure()
ax = plt.axes(projection='3d')
ax.plot_surface(X, Y, Z, rstride=1, cstride=1,cmap='jet', edgecolor='none')
```

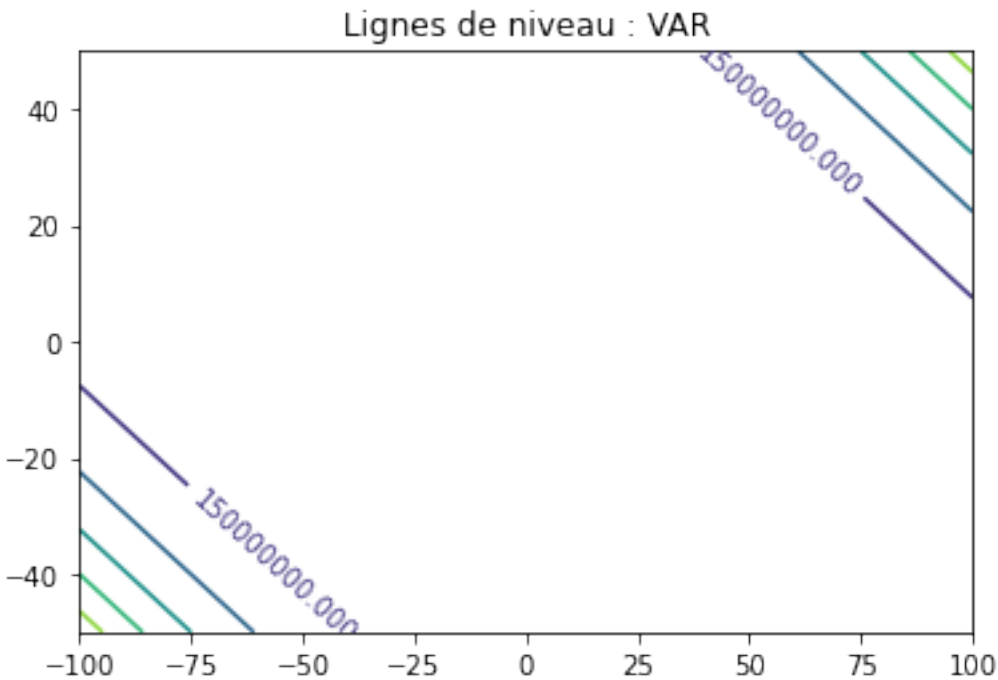
[89]:
```
<mpl_toolkits.mplot3d.art3d.Poly3DCollection at 0x1e1b1de31c0>
```



Lignes de niveau

[90]:
```
# Allure des lignes de niveau de la fonction VAR
import numpy as np
import matplotlib.pyplot as plt

xmin = -100
xmax = 100
nbx = 200
ymin = -50
ymax = 50
nby = 200

x = np.linspace(xmin, xmax, nbx)
y = np.linspace(ymin, ymax, nby)
X, Y = np.meshgrid(x, y)

x=np.array([X,Y])

Z = VAR_dim2(x)

fig, ax = plt.subplots()
CS = ax.contour(X, Y, Z)
ax.clabel(CS, inline=True, fontsize=10)
ax.set_title('Lignes de niveau : VAR')
```

[90]: Text(0.5, 1.0, 'Lignes de niveau : VAR')



Résultats avec l'algorithme SDM

[91]:
```
# Test de l'algorithme SDM
n=3 # Dimension du vecteur
np.random.seed(10)
x=np.random.sample(n) # random number
x,gradf,it,vect=SDM(VAR,grad_Var,x)
print ("Résulat de l'algorithme SDM : x=",x)
print ("avec un gradient de norme %f, après %d itération et une erreur %f" %(np.
↪linalg.norm(gradf),it,vect[-1]))
```

```
Résulat de l'algorithme SDM : x= [-3.59115675e-11  1.70863975e-11
6.16666898e-12]
avec un gradient de norme 0.000000, après 30 itération et une erreur 0.000000
```

L'algorithme converge bien ici.

[92]:
```
# Test de l'algorithme FR
n=3 # Dimension du vecteur
np.random.seed(10)
x=np.random.sample(n) # random number
x,r,it,vect=FR(VAR,grad_Var,x)
print ("Résulat de l'algorithme FR : x=",x)
print ("avec un gradient de norme %f, après %d itération et une erreur %f" %(np.
↪linalg.norm(r),it,vect[-1]))
```

```
Résulat de l'algorithme FR : x= [-1.94160401e-11  1.84668237e-11
1.91147321e-13]
avec un gradient de norme 0.000000, après 29 itération et une erreur 0.000000
```

L'algorithme converge également.

Comparaison des deux algorithmes

[93]:
```
comparaison(50,VAR,grad_Var)
```

```
l algorithme SDM n'a pas convergé : 0/50
nombre moyen d itérations pour SDM : 18

l algorithme FR n'a pas convergé : 0/50
nombre moyen d itérations pour FR : 14
```

Le nombre d'itérations moyen des deux algorithmes est sensiblement le même.

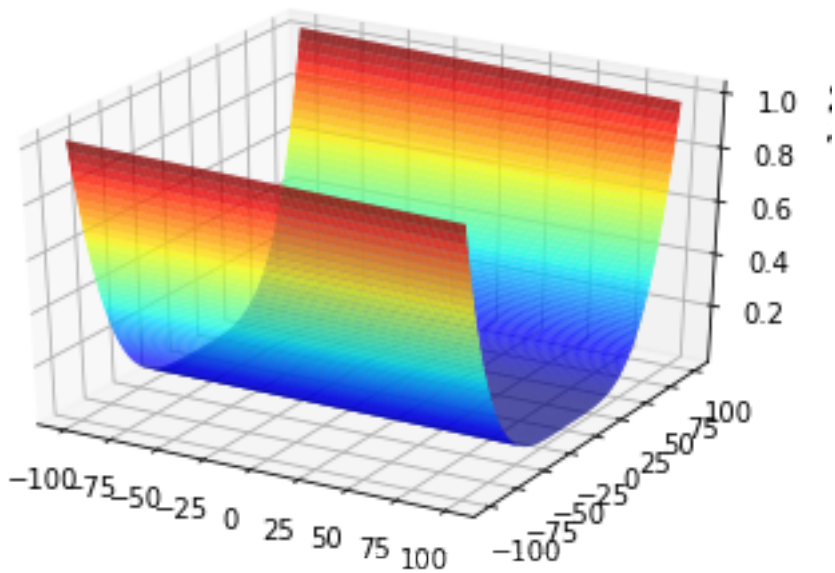
1.6 Fonction MONDIA

[94]:
```
# En dimension 2
def Mondia(x):
    x1=x[0]
    x2=x[1]
    f=100*(x1-x2**2)**2+(1-x1)**2+(1-x2)**2
    return f

def grad_Mond(x):
    x1=x[0]
    x2=x[1]

    y = np.asarray(x)
    gradf = np.zeros_like(y)
    gradf[0]=200*(x1-x2**2)+2*(x1-1)
    gradf[1]=400*x2*(x2**2-x1)

    return gradf
```

[95]:
```
# Représentation graphique de la fonction Mondia
import numpy as np
import matplotlib.pyplot as plt

xmin = -100
xmax = 100
nbx = 200
ymin = -100
ymax = 100
nby = 200
```

```
x = np.linspace(xmin, xmax, nbx)
y = np.linspace(ymin, ymax, nby)
X, Y = np.meshgrid(x, y)

x=np.array([X,Y])

Z = Mondia(x)

fig = plt.figure()
ax = plt.axes(projection='3d')
ax.plot_surface(X, Y, Z, rstride=1, cstride=1,cmap='jet', edgecolor='none')
```

[95]: <mpl_toolkits.mplot3d.art3d.Poly3DCollection at 0x1e1ac833cd0>

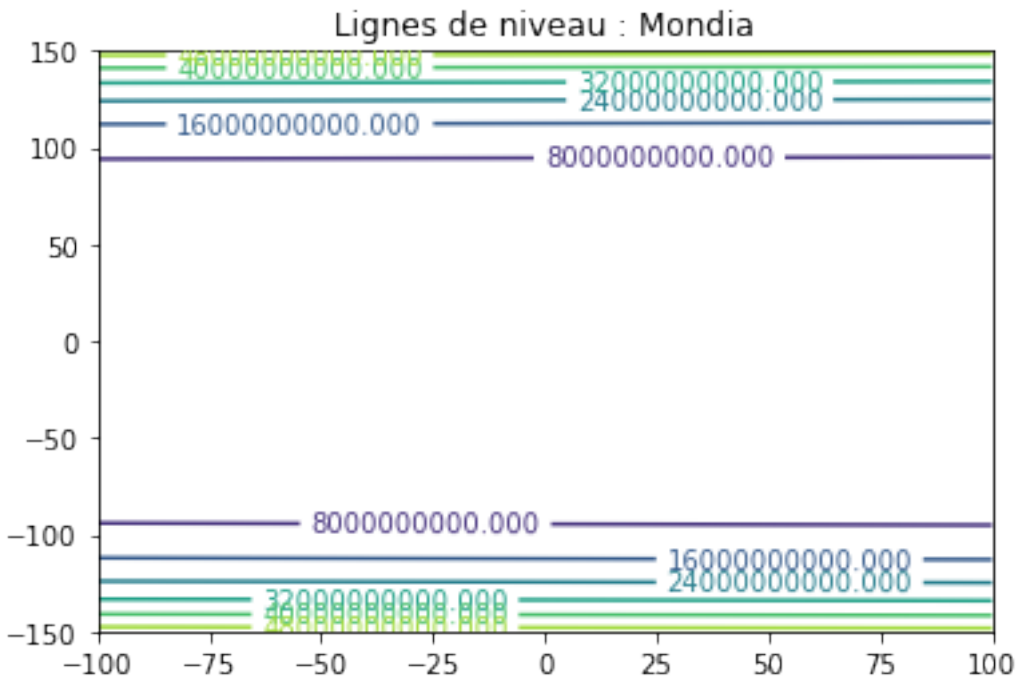


1.7 Lignes de niveaux

[103]:
```
# Allure des lignes de niveau de la fonction Mondia
import numpy as np
import matplotlib.pyplot as plt

xmin = -100
xmax = 100
nbx = 200
ymin = -150
ymax = 150
nby = 200
```

```
x = np.linspace(xmin, xmax, nbx)
y = np.linspace(ymin, ymax, nby)
X, Y = np.meshgrid(x, y)

x=np.array([X,Y])

Z = Mondia(x)

fig, ax = plt.subplots()
CS = ax.contour(X, Y, Z)
ax.clabel(CS, inline=True, fontsize=10)
ax.set_title('Lignes de niveau : Mondia')
```

[103]: Text(0.5, 1.0, 'Lignes de niveau : Mondia')



Résultat de l'algorithme SDM

[96]:
```
# Test de l'algorithme SDM
np.random.seed(10)
x=np.random.sample(2) # random number
x,gradf,it,vect=SDM(Mondia,grad_Mond,x)
print ("Résulat de l'algorithme SDM : x=",x)
print ("avec un gradient de norme %f, après %d itération et une erreur %f" %(np.
↪linalg.norm(gradf),it,vect[-1]))
```

```
Résulat de l'algorithme SDM : x= [1. 1.]
avec un gradient de norme 0.000000, après 709 itération et une erreur 0.000000
```

L'algorithme converge en 709 itérations.

Résultats de l'algorithme FR

[97]:
```
# Test de l'algorithme FR
np.random.seed(10)
x=np.random.sample(2) # random number
x,r,it,vect=FR(Mondia,grad_Mond,x)
print ("Résulat de l'algorithme FR : x=",x)
print ("avec un gradient de norme %f, après %d itération et une erreur %f" %(np.
 ↪linalg.norm(r),it,vect[-1]))
```

```
Résulat de l'algorithme FR : x= [1. 1.]
avec un gradient de norme 0.000000, après 76 itération et une erreur 0.000000
```

L'algorithme converge en 76 itérations.

1.7.1 Comparaison des deux algorithmes

[98]:
```
comparaison(50,Mondia,grad_Mond)
```

```
l algorithme SDM n'a pas convergé : 0/50
nombre moyen d itérations pour SDM : 447

l algorithme FR n'a pas convergé : 0/50
nombre moyen d itérations pour FR : 46
```

Les deux algorithmes convergent mais celui de Fletcher le fait en moins d'itération.

1.8 Conclusion

Les deux algorithmes amènent globalement à trouver la solution (même si la tolérance de 10^-10 peut être trop petite parfois).

L'algorithme de Fletcher a dans la plupart des cas besoin d'effectuer moins d'itération que celui de SDM.

[ ]:
```

```

TP2 exercice 4

March 31, 2021

0.1 Exo 4

0.2 Second order Newton type methods

First of all we write two Newton methods: a classical one and another called *BFGS*, then we are goint to test theses methods with several functions.

Remark: The input should be an intial point and a funcion with its gradient and Hessian

0.2.1 Classical Newton method

[74]:
```
from numpy.linalg import solve, norm

def ClassicalNewton(g, h, x, tol, itmax):

    it = 0
    vect = [0]

    while norm(g(x)) > tol and it < itmax:

        it += 1
        d = -solve(h(x), g(x))#solve the linear system
        p = 1
        x = x + p * d
        vect = append(vect, norm(g(x)))

    return x, g(x), it, vect
```

Remark: Here we use a *p(stepsize)* which equals to 1, but we can use $p =$ *Armijo methods* to change the method. It is also called *"DampedNewton method"*.

0.2.2 The well known BFGS Method

[75]:
```
from numpy import dot, asarray, mat, append
from numpy.linalg import norm

#we have to define the Armijo method first
def Armijo(f, g0, x, d, t = 1, beta = 0.5, alpha = 0.25, itmax = 10000):
    it = 0
    while (f(x + t*d) > f(x) + alpha*t*dot(g0, d)) and it < itmax:
```

```
        it += 1
        t = beta*t
    return t, f(x + t*d), it

def BFGS(f, g, H, x, tol, itmax):
    it = 0
    vect = [0]
    while norm(g(x)) > tol and it < itmax:
        it += 1
        d = -dot(H, g(x)) #compute d = -H*g
        rho = Armijo(f, g(x), x, -g(x))[0] # compute the stepsizeby minimising␣
↪or using Armijo

        p = rho*d
        q = g(x + p) - g(x)

        pp = asarray(mat(p).T*mat(p))
        pq = asarray(mat(p).T*mat(q))
        qp = asarray(mat(q).T*mat(p))

        H = H + (1 + q.dot(H).dot(q)/q.dot(p))*pp/p.dot(q)- (pq.dot(H) + H.
↪dot(qp))/q.dot(p)
        #update the matrix H using the CR2 formula

        x = x + p
        vect = append(vect, norm(g(x)))
    return x, g(x), it, vect
```

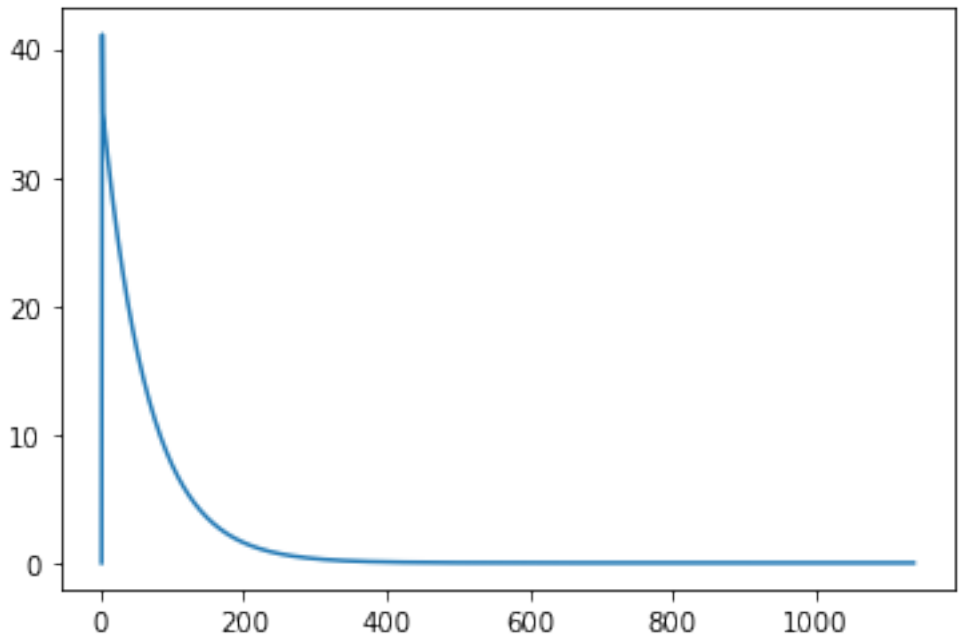
After writing these two methods, we are goint to test them with following functions:

1. $ f(x) = x_1^2 +6x_1x_2+25x_2^2-12x_1-2x_2-6$

[77]:
```
# f1
from matplotlib.pyplot import plot, show
from numpy import zeros, array,eye,linspace

tol = 1e-10
itmax = 10000

f = lambda x: x[0]**2 + 6*x[0]*x[1] + 25*x[1]**2 - 12*x[0] - 2*x[1] - 6
g = lambda x: array([2*x[0] + 6*x[1] - 12 , 6*x[0] + 50*x[1] - 2])
h = lambda x: array([[2, 6], [6, 50]])

#using classical Newton
x, r, it, vect = ClassicalNewton(g, h, zeros(2),tol,itmax)

print ("Classical Newton Result:")
print ("x: ", x)
```

```
print ("Norm of Gradient: %f" % norm(r))
print ("Number of iterations: %d \nError: %f" % (it, vect[-1]))

print("_____________________________________________________________")

#Using BFGS
x, r, it, vect = BFGS(f, g, eye(2), zeros(2),tol,itmax)

print (" ")
print ("BFGS Result:")
print ("x: ", x)
print ("Norm of Gradient: %f" % norm(r))
print ("Number of iterations: %d \nError: %f" % (it, vect[-1]))

plot(linspace(1,it,num=len(vect)),vect)
show()
```

```
Classical Newton Result:
x:  [ 9.1875 -1.0625]
Norm of Gradient: 0.000000
Number of iterations: 1
Error: 0.000000
------------------------------------------------------------

BFGS Result:
x:  [ 9.1875 -1.0625]
Norm of Gradient: 0.000000
Number of iterations: 1138
Error: 0.000000
```



We can easily find that the second order method is obviously faster than the descent method. But why ? Becaue Newton's method is second-order convergence, and gradient descent is first-order convergence, so Newton's method is faster. To put it more simplistically, for example, if you want to find the shortest path to the bottom of a basin, the gradient descent method only chooses one step in the direction with the largest slope from your current position at a time, and Newton's method when choosing the direction , Will not only consider whether the slope is large enough, but also whether the slope will become larger after you take a step. Therefore, it can be said that Newton's method looks a bit farther than the gradient descent method and can reach the bottom faster.

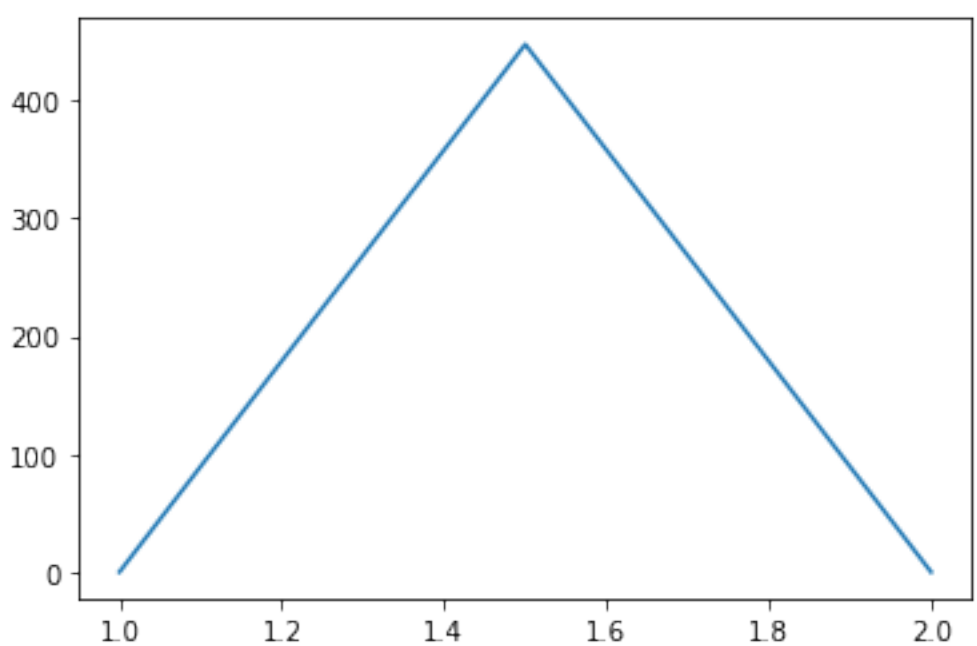
And here comes another example which is called Rosenbrock's function:

2. $f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$

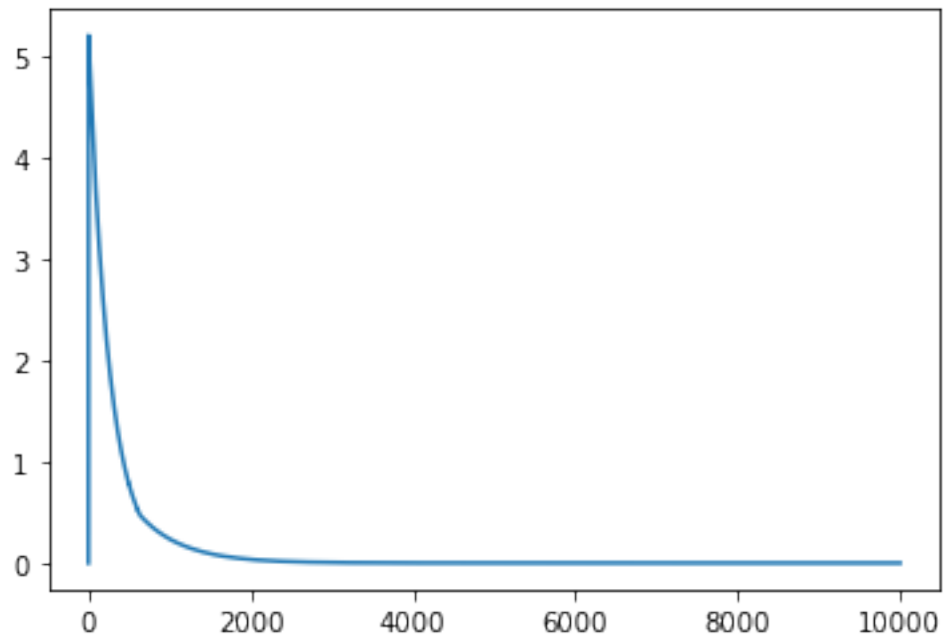
[79]:
```
f = lambda x: 100*(x[1] - x[0]**2)**2 + (1 - x[0])**2
g = lambda x: array([400*x[0]**3 -400*x[0]*x[1]+2*x[0]- 2 , -200*(x[0]**2) +␣
↪200*x[1]])
h = lambda x: array([[1200*x[0]**2-400*x[1]+2, -400*x[0]], [-400*x[0], 200]])

#Using classical Newton
x, r, it, vect = ClassicalNewton(g, h, zeros(2),tol,itmax)

print ("Classical Newton Result:")
print ("x: ", x)
print ("Norm of Gradient: %f" % norm(r))
print ("Number of iterations: %d \nError: %f" % (it, vect[-1]))
```

```
plot(linspace(1,it,num=len(vect)),vect)
show()

print("_____________________________________")
print(" ")

#Using BFGS
x, r, it, vect = BFGS(f, g, eye(2), zeros(2),tol,itmax)

print (" ")
print ("BFGS Result:")
print ("x: ", x)
print ("Norm of Gradient: %f" % norm(r))
print ("Number of iterations: %d \nError: %f" % (it, vect[-1]))

plot(linspace(1,it,num=len(vect)),vect)
show()
```

```
Classical Newton Result:
x:  [1. 1.]
Norm of Gradient: 0.000000
Number of iterations: 2
Error: 0.000000
```



```
BFGS Result:
x:  [1.         0.99999999]
Norm of Gradient: 0.000000
Number of iterations: 10000
Error: 0.000000
```



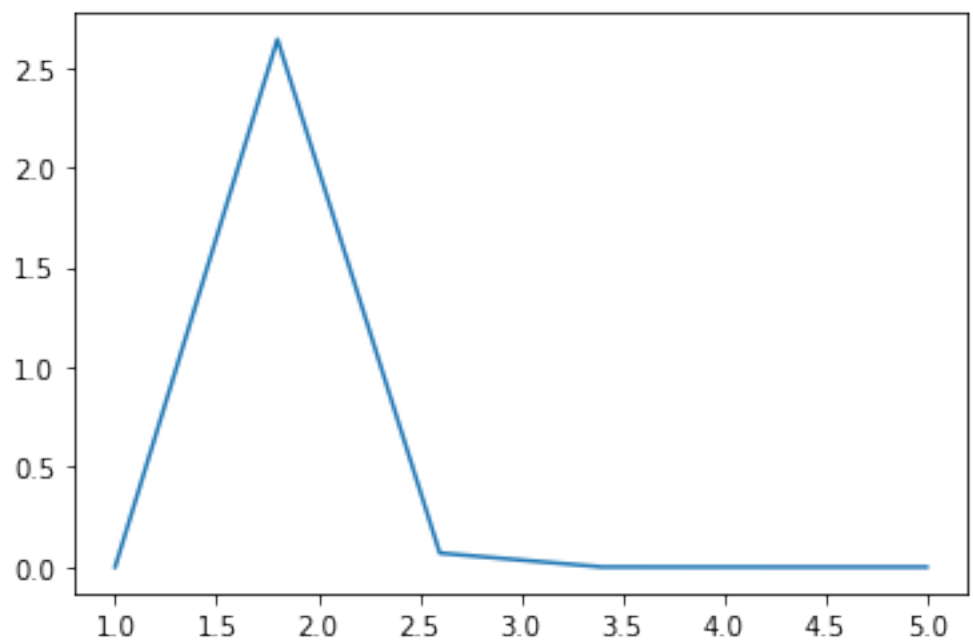
Then we will test Himmelblau's function written in this form:

3. $f(x_1, x_2) = (x_2 + x_1^2 - 11)^2 + (x_1 + x_2^2 - 7)^2$

[81]:
```
f = lambda x: (x[1]+x[0]**2-11)**2+(x[0]+x[1]**2-7)**2
g = lambda x: array([4*x[0]*x[1]+4*x[0]**3-44*x[0]+2*(x[0]+x[1]**2-7) ,
                     2*(x[1]+x[0]**2-11)+4*x[0]*x[1]+4*x[1]**3-28*x[1]])
h = lambda x: array([[4*x[1]+12*x[0]**2-42 , 4*x[0]+4*x[1]],
                     [4*x[0]+4*x[1], 4*x[0]+12*x[1]**2-26]])

#Using classical Newton
x, r, it, vect = ClassicalNewton(g, h, zeros(2),tol,itmax)

print ("Classical Newton Result:")
print ("x: ", x)
print ("Norm of Gradient: %f" % norm(r))
print ("Number of iterations: %d \nError: %f" % (it, vect[-1]))

plot(linspace(1,it,num=len(vect)),vect)
show()
```

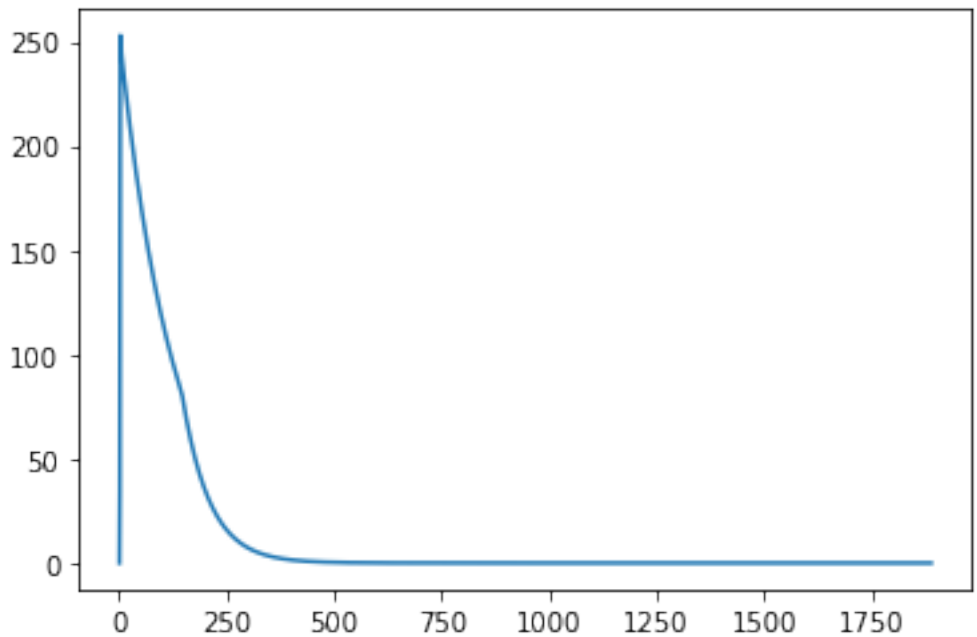
```
print("_____________________________________")
print(" ")

#Using BFGS
x, r, it, vect = BFGS(f, g, eye(2), zeros(2),tol,itmax)

print (" ")
print ("BFGS Result:")
print ("x: ", x)
print ("Norm of Gradient: %f" % norm(r))
print ("Number of iterations: %d \nError: %f" % (it, vect[-1]))

plot(linspace(1,it,num=len(vect)),vect)
show()
```

```
Classical Newton Result:
x:  [-0.27084459 -0.92303856]
Norm of Gradient: 0.000000
Number of iterations: 5
Error: 0.000000
```




```
_____________________________________

BFGS Result:
```

```
x:  [3. 2.]
Norm of Gradient: 0.000000
Number of iterations: 1888
Error: 0.000000
```



Further more, we choose the dimension n to be 3 in order to simplify the tests of theses following functions.

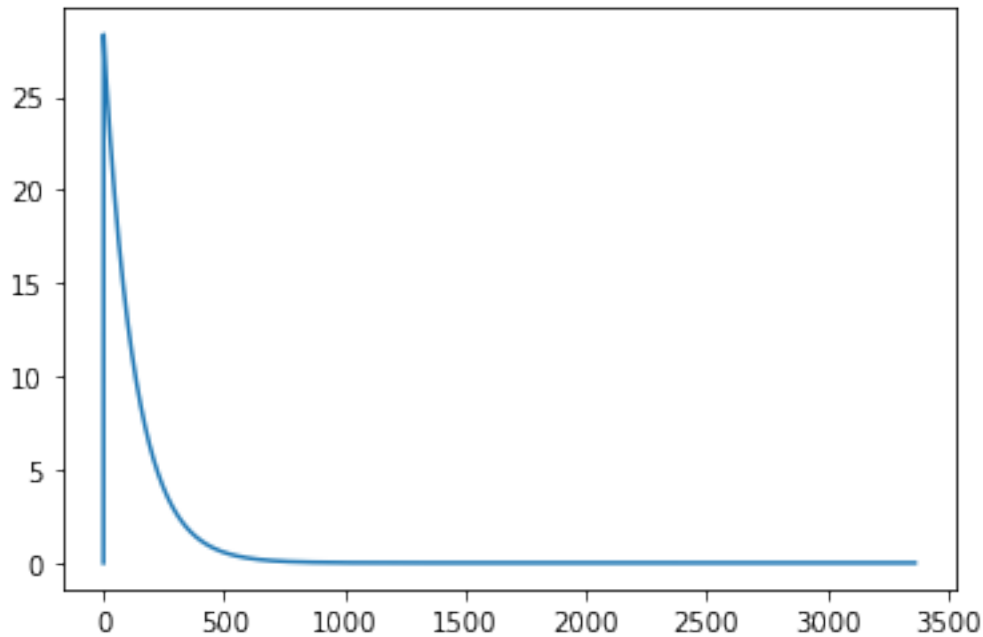
The first one should be the DENNI's function.

$F(x) = \sum_{1\leq k\leq n} kx_k^2 + (\sum_{1\leq k\leq n} x_k)^2$

[83]:
```
f = lambda x: sum( (i-1)*x[i-1]**2 for i in range(1,3) ) + (sum( x[i-1] for i␣
↪in range(1,3) ))**2
#index in python begins with 0

g = lambda x: array([4*x[0] + 2*x[1] + 2*x[2],
                     2*x[0] + 6*x[1] + 2*x[2],
                     2*x[0] + 2*x[1] + 8*x[2]])
h = lambda x: array([[4,2,2],
                     [2,6,2],
                     [2,2,8]])

#Using classical Newton
x, r, it, vect = ClassicalNewton(g, h, zeros(3)+eye(3),tol,itmax)

print ("Classical Newton Result:")
```

```
print ("x: ", x)
print ("Norm of Gradient: %f" % norm(r))
print ("Number of iterations: %d \nError: %f" % (it, vect[-1]))

print("_____________________________________")
print(" ")

#Using BFGS
x, r, it, vect = BFGS(f, g, eye(3), array([1,2,2]),tol,itmax)

print (" ")
print ("BFGS Result:")
print ("x: ", x)
print ("Norm of Gradient: %f" % norm(r))
print ("Number of iterations: %d \nError: %f" % (it, vect[-1]))

plot(linspace(1,it,num=len(vect)),vect)
show()
```

```
Classical Newton Result:
x:  [[0.00000000e+00 0.00000000e+00 0.00000000e+00]
 [0.00000000e+00 0.00000000e+00 0.00000000e+00]
 [0.00000000e+00 8.16340459e-18 0.00000000e+00]]
Norm of Gradient: 0.000000
Number of iterations: 1
Error: 0.000000
_____________________________________


BFGS Result:
x:  [3.17001991e-12 6.50841883e-12 6.41256366e-12]
Norm of Gradient: 0.000000
Number of iterations: 3363
Error: 0.000000
```



- *remark 1: Sometimes we can't even have a proper plot, assuming that the codes should be typed correctly, the reason is supposed to be the wrong intial condition while looping in the method.*
- *remark 2: We can't start the x_0 with (0,0,0) which is already the solution*

Then we test VAR's function :

$F(x) = \sum_{1\leq k\leq n} x_k^2 + (\sum_{1\leq k\leq n} \sqrt{k}x_k)^2 + (\sum_{1\leq k\leq n} \sqrt{k}x_k)^4$

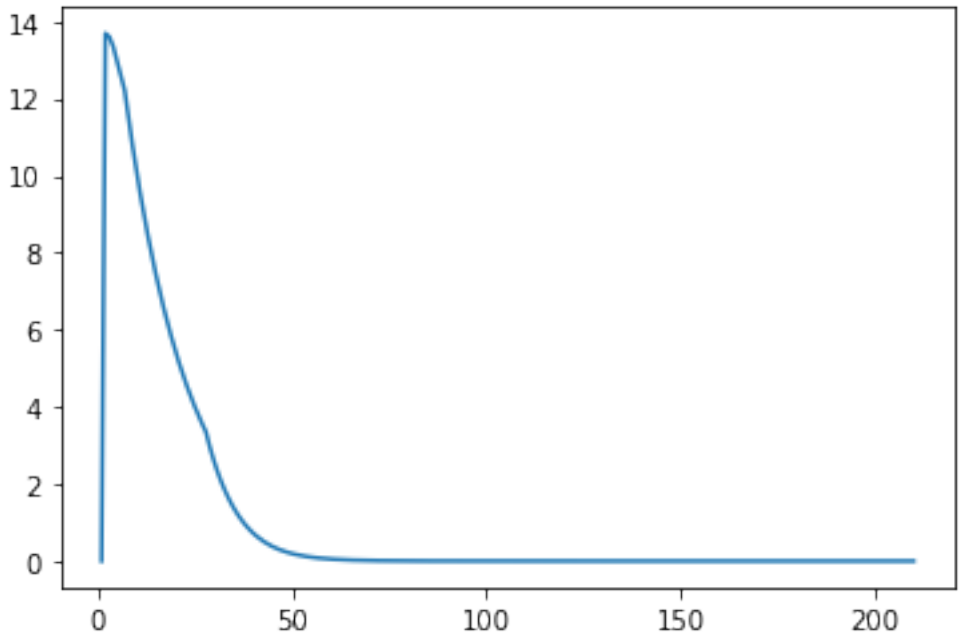
[84]:
```
from numpy import sqrt
import numpy as np

def VAR(x):
    x1=x[0]
    x2=x[1]
    f=x1**2+x2**2+(x1+sqrt(2)*x2)**2+(x1+np.sqrt(2)*x2)**4
    return f

def g_var(x):
    x1=x[0]
    x2=x[1]
    gradf=array([2*x1+2*(x1+sqrt(2)*x2)+4*(x1+sqrt(2)*x2)**3,
                 2*x2+2*sqrt(2)*(x1+sqrt(2)*x2)+4*sqrt(2)*(x1+sqrt(2)*x2)**3])
    return gradf

def h_var(x):
    x1=x[0]
```

```
    x2=x[1]
    hf=array([[4+12*(x1+sqrt(2)*x2)**2,␣
↪2*sqrt(2)+12*sqrt(2)*(x1+sqrt(2)*x2)**2],
             [2*sqrt(2)+12*sqrt(2)*(x1+sqrt(2)*x2)**2,␣
↪6+24*(x1+sqrt(2)*x2)**2]])
    return hf

#Using classical Newton
x, r, it, vect = ClassicalNewton(g_var, h_var, zeros(2)+eye(2),tol,itmax)

print ("Classical Newton Result:")
print ("x: ", x)
print ("Norm of Gradient: %f" % norm(r))
print ("Number of iterations: %d \nError: %f" % (it, vect[-1]))

print("_____________________________________")
print(" ")

#Using BFGS
x, r, it, vect = BFGS(VAR, g_var, eye(2), array([1,2]),tol,itmax)

print (" ")
print ("BFGS Result:")
print ("x: ", x)
print ("Norm of Gradient: %f" % norm(r))
print ("Number of iterations: %d \nError: %f" % (it, vect[-1]))

plot(linspace(1,it,num=len(vect)),vect)
show()
```

```
Classical Newton Result:
x:  [[nan nan]
 [nan nan]]
Norm of Gradient: nan
Number of iterations: 10
Error: nan
-------------------------------------


BFGS Result:
x:  [5.43689034e-12 1.01651810e-11]
Norm of Gradient: 0.000000
Number of iterations: 210
Error: 0.000000
```

```
<ipython-input-84-0947e8ab1770>:13: RuntimeWarning: overflow encountered in
power
  gradf=array([2*x1+2*(x1+sqrt(2)*x2)+4*(x1+sqrt(2)*x2)**3,
```

```
<ipython-input-84-0947e8ab1770>:14: RuntimeWarning: overflow encountered in
power
  2*x2+2*sqrt(2)*(x1+sqrt(2)*x2)+4*sqrt(2)*(x1+sqrt(2)*x2)**3])
```



remark : The initial point needs to be sufficiently "close" to the minimum point, otherwise, the algorithm may not converge. If not, we may never have the convergence

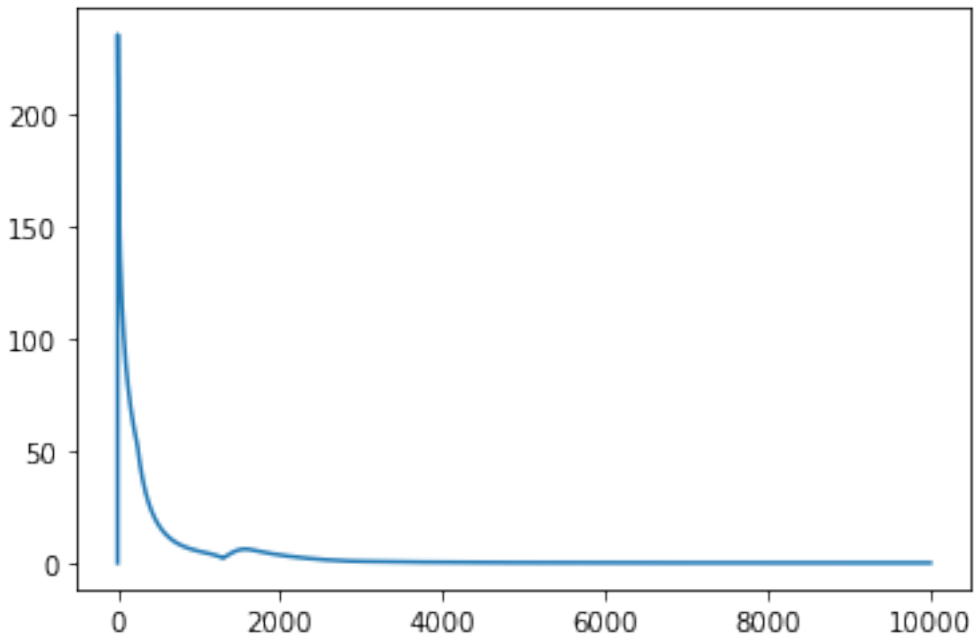
The last one is MONDIA's function

$F(x) = \sum_{2\leq k\leq n} 100(x_1 - x_k^2)^2 + \sum_{1\leq k\leq n}(1 - x_k)^2$

[85]:
```
def f(x):
    x1=x[0]
    x2=x[1]
    f = 100*(x1-x2**2)**2+(1-x2)**2+(1-x1)**2
    return f

def g(x):
    x1=x[0]
    x2=x[1]
    gradf=array([200*(x1-x2**2)+2*(x1-1),
                 400*x2**3-400*x1*x2+2*x2-2])
    return gradf

def h(x):
    x1=x[0]
    x2=x[1]
```

```
    hf=array([[202,-400*x2],
                 [-400*x2,1200*x2**2-400*x1+2]])
    return hf

'''
#error in using classical Newton,particularly in solving the linear system
x, r, it, vect = ClassicalNewton(g, h, eye(2),tol,itmax)

print ("Classical Newton Result:")
print ("x: ", x)
print ("Norm of Gradient: %f" % norm(r))
print ("Number of iterations: %d \nError: %f" % (it, vect[-1]))

'''
print("_____________________________________")
print(" ")

#Using BFGS
x, r, it, vect = BFGS(f, g, eye(2), array([1,2]),tol,itmax)

print (" ")
print ("BFGS Result:")
print ("x: ", x)
print ("Norm of Gradient: %f" % norm(r))
print ("Number of iterations: %d \nError: %f" % (it, vect[-1]))

plot(linspace(1,it,num=len(vect)),vect)
show()
```

```
-------------------------------------

<ipython-input-85-92fcf202dbfe>:4: RuntimeWarning: overflow encountered in
long_scalars
  f = 100*(x1-x2**2)**2+(1-x2)**2+(1-x1)**2


BFGS Result:
x:  [0.99986558 0.99993356]
Norm of Gradient: 0.000754
Number of iterations: 10000
Error: 0.000754
```



Noted that the number of iterations is already 10000 that breaks out the loop, we don't have the convergence in this case.

Conclusion :

- After analysing the results that we have already obtained, we found that the BFGS is believed to be stable and has the best scheme. we also have *quadratic convergent rate* with Classical-Newton method, and with BFGS, we have *superlinear convergent rate*. Which corresponds to the course that we learnt.
- Using the $ClassicalNewton$ has the faster convergence than $BFGS$. But the problem is that we can't always easily calculate the Hessian Matrix. In low dimension it's ok, unluckily, the higher dimension we have, the more expensive coste to calculate the Hessian. And we also have bugs in solve(Hessian,Gradiant) while coding. Anyway it takes plenty of time in reality to solve the problem. It's quite rare to use this method pratically. Therefore, sometimes we use Quasi-newton (AKA $BFGS$) instead. It allows us to changing the H_k at each iteration.
- To go further, if we search quasi_newton method in internet, we can also find The $DFP $ method and $Limited - memoryBFGS$ method when we have a high dimension matrix, we applied L-BFGS in order to limit cost in storaging the memory.
- Considering the pratical use, in deep learning for example, the gradient descent method is often used as the optimization operator, and the Newton method is rarely used. The main reasons are as follows:
  - Neural networks are usually non-convex. In this case, the convergence of Newton's method is difficult to guarantee
  - Even with convex optimization, Newton's method will show the advantage of fast convergence only when the iteration point is very close to the global optimum

- May be attracted by saddle points